

claude-windows / 7fe4ef45-d30a-448e-98eb-4da4fba244f1

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7fe4ef45-d30a-448e-98eb-4da4fba244f1\subagents\workflows\wf\_b531eaeef-39c\agent-a0bf0314fe07b6894.jsonl`
- SHA-256: `ab413659372ca40406d92eee2e031091448026d534297f9355e05bbb1e743fb8`
- Source modified: `2026-06-19T17:03:00+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf\_b531eaeef-39c`
- session_id: `7fe4ef45-d30a-448e-98eb-4da4fba244f1`

Transcript

1. user (2026-06-19T16:59:09.587Z)

You are an adversarial design reviewer. Be skeptical; the goal is to find real flaws BEFORE we build.

CONTEXT ? badminton/racket-sports rally detector. Pipeline: WASB shuttle DETECTOR (heatmap ? per-frame candidate list) ? single-target online TRACKER (collapses to ONE (x,y) per frame) ? velocity windower ? rallies.

MEASURED (GX010142, owner's anchor clip, on the 21-25s window, from the cached det_raw.jsonl detector candidates):

- WASB's detector emits ≥ 2 candidates on 31% of detected frames (cached, never used past the tracker).
- During the 21-25s rally: 93% of frames carry the ADJACENT-court shuttle (x-norm ~ 0.88 , what the tracker followed),
 - 83% ALSO carry a PROMINENT-court shuttle (x-norm ~ 0.56), 76% carry BOTH at once.
- A greedy nearest-neighbour reconstruction of the prominent-court candidates yields a CONTINUOUS rally arc
 - (75% window coverage, 431px vertical excursion, 8.6px median per-frame jump, ~ 2.4 s sustained)

? VISUALLY CONFIRMED

as a real foreground-court rally (player mid-stroke ? net ? second player), which the single-target tracker discarded.

PROPOSED DESIGN ("the Which problem"):

Split the tracker's two conflated jobs:

- Layer 1 "see everything": a MULTI-TARGET linker over the ALREADY-CACHED det_raw.jsonl candidates ? a SET of shuttle
 - tracks per region (top-K), each with trajectory + duration + motion-energy. Pure-Python over the cache, ZERO GPU,
 - WASB stays frozen (MIT).
- Layer 2 "pick the match's story": score each track by court-membership (the existing prominent-court polygon +
 - point_in_polygon) + rally-likeness (motion energy, vertical excursion, ≥ 2 net-crossings/Gate-A, ends on shuttle-floor-contact) and SELECT the track(s) whose story is the match's; feed those to the windower.

Claimed benefits: recall-POSITIVE (recovers the discarded prominent rally ? a G1 miss) AND precision-positive (drops the adjacent rally) ? unlike today's court_gate which is a subtractive post-filter on the single track (precision-only). Also claimed to SUBSUME the "ghost" (a stuck-pixel false-lock track has ~ 0 motion energy ? never selected). Lands default-OFF, flag-gated, measured on cached candidates + golden labels with no GPU; single-track tracker stays the default.

YOUR LENS:

Attack the claim that multi-track + motion-energy selection SUBSUMES the "ghost" (stuck-pixel false-lock: GX010128 had 284 frames at exactly x=1680px; Badminton_BXH_2 ~41% of points parked at court-center). Does a low-motion-energy filter actually kill these, or can a ghost still win selection (e.g. it sits INSIDE the prominent-court polygon, or a real rally is ALSO present and the ghost fragments it)? Edge cases where the ghost survives. Mitigations. Also: is the ghost truly the same root cause (selection) as multi-shuttle, or a distinct detector bug that needs its own fix?

Return concrete, technically-grounded risks with severity + mitigations. Mark kills_design=true ONLY for a fundamental unmitigable flaw.

2. assistant (2026-06-19T16:59:12.429Z)

I'll investigate this thoroughly before forming a verdict. Let me look at the actual code and data to ground my analysis of the ghost-subsumption claim.

3. user (2026-06-19T16:59:16.188Z)

Your configuration specifies to merge with the ref 'refs/heads/docs/court-floor-rally-end' from the remote, but no such ref was fetched.

---BRANCH---

docs/court-floor-rally-end

4. user (2026-06-19T16:59:16.403Z)

backend\detectors__pycache__\rally_gate.cpython-313.pyc
tests__pycache__\test_rally_gate.cpython-313.pyc
backend\pipeline\detectors\rally_gate.py
tests\test_rally_gate.py
tests__pycache__\test_rally_gate.cpython-313-pytest-9.0.3.pyc
.claude\worktrees\bhi-execpolicy\backend\pipeline\detectors\rally_gate.py
.claude\worktrees\bhi-execpolicy\tests\test_rally_gate.py
.claude\worktrees\bhi-execpolicy\tests__pycache__\test_rally_gate.cpython-313-pytest-9.0.3.pyc
.claude\worktrees\bhi-execpolicy\backend\pipeline\detectors__pycache__\rally_gate.cpython-313.pyc
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\pipeline\detectors\rally_gate.py
.claude\worktrees\agent-a771ff1d85e18f2ed\tests\test_rally_gate.py
.claude\worktrees\agent-a771ff1d85e18f2ed\tests__pycache__\test_rally_gate.cpython-313-pytest-9.0.3.pyc
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\pipeline\detectors__pycache__\rally_gate.cpython-313.pyc
.claude\worktrees\tier1-windowing\backend\pipeline\detectors\rally_gate.py
.claude\worktrees\tier1-windowing\tests\test_rally_gate.py
.claude\worktrees\tier1-windowing\tests__pycache__\test_rally_gate.cpython-313-pytest-9.0.3.pyc
.claude\worktrees\tier1-windowing\backend\pipeline\detectors__pycache__\rally_gate.cpython-313.pyc
.claude\worktrees\w2-safeguard\backend\pipeline\detectors\rally_gate.py
.claude\worktrees\w2-safeguard\tests\test_rally_gate.py
.claude\worktrees\w2-safeguard\tests__pycache__\test_rally_gate.cpython-313-pytest-9.0.3.pyc
.claude\worktrees\w2-safeguard\backend\pipeline\detectors__pycache__\rally_gate.cpython-313.pyc
.claude\worktrees\hardchop\backend\pipeline\detectors\rally_gate.py
.claude\worktrees\hardchop\tests\test_rally_gate.py
.claude\worktrees\hardchop\tests__pycache__\test_rally_gate.cpython-313-pytest-9.0.3.pyc
.claude\worktrees\hardchop\backend\pipeline\detectors__pycache__\rally_gate.cpython-313.pyc
.claude\worktrees\boxhill17\backend\pipeline\detectors\rally_gate.py
.claude\worktrees\boxhill17\tests\test_rally_gate.py
.claude\worktrees\boxhill17\backend\pipeline\detectors__pycache__\rally_gate.cpython-313.pyc
.claude\worktrees\pr-stitch\backend\pipeline\detectors\rally_gate.py
.claude\worktrees\pr-stitch\tests\test_rally_gate.py
.claude\worktrees\pr-stitch\tests__pycache__\test_rally_gate.cpython-313-pytest-9.0.3.pyc
.claude\worktrees\pr-stitch\backend\pipeline\detectors__pycache__\rally_gate.cpython-313.pyc
.claude\worktrees\gap-doc\backend\pipeline\detectors\rally_gate.py
.claude\worktrees\gap-doc\tests\test_rally_gate.py

```
.claude\worktrees\resweep\backend\pipeline\detectors\rally_gate.py
.claude\worktrees\resweep\tests\test_rally_gate.py
.claude\worktrees\recdoc\backend\pipeline\detectors\rally_gate.py
.claude\worktrees\recdoc\tests\test_rally_gate.py
backend\pipeline\detectors\__pycache__\rally_gate.cpython-313.pyc
```

5. user (2026-06-19T16:59:16.412Z)

```
.git\refs\remotes\origin\docs\prominent-court-design
.git\logs\refs\remotes\origin\docs\prominent-court-design
backend\pipeline\detectors\court_gate.py
tests\test_court_gate.py
tests\__pycache__\test_court_gate.cpython-313-pytest-9.0.3.pyc
.git\logs\refs\heads\feat\prominent-court-gate
.git\refs\heads\feat\prominent-court-gate
.git\refs\remotes\origin\feat\prominent-court-gate
.git\logs\refs\remotes\origin\feat\prominent-court-gate
backend\pipeline\detectors\__pycache__\court_gate.cpython-313.pyc
output\_suite_courtgate.log
.git\refs\heads\docs\court-floor-rally-end
.git\logs\refs\heads\docs\court-floor-rally-end
.git\refs\remotes\origin\docs\court-floor-rally-end
.git\logs\refs\remotes\origin\docs\court-floor-rally-end
output\multicourt_analysis.md
```

6. assistant (2026-06-19T16:59:20.032Z)

The branch tracks a remote ref that wasn't fetched (likely deleted upstream). That's a session-hygiene note, not relevant to my task. Let me proceed read-only and dig into the actual code and data.

7. user (2026-06-19T16:59:20.983Z)

```
1      """Prominent-court shuttle gate ? multicourt G2 (see
2      ``docs/PROMINENT_COURT_DETECTION.md``).
3
4      In multicourt footage the WASB shuttle tracker follows ANY shuttle in frame, so a rally
5      on a
6      background / adjacent court becomes a false rally on the filmed match. This gate re-
7      defines the
8      shuttle signal ``(x, y, visible)`` ? ``(x, y, visible AND inside-the-prominent-court)``:
9      a visible
10     shuttle point whose normalised position is outside the prominent-court polygon is marked
11     NOT
12     visible, so the velocity windower never opens a rally there.
13
14     Pure (no GPU / cv2): point-in-polygon over the court polygon (reuses
15     ``fusion_features.point_in_polygon``). **Subtractive only** ? it can only CLEAR
16     visibility, never
17     set it ? so on a correct polygon it suppresses out-of-court rallies without
18     manufacturing any.
19     ``polygon=None`` (or < 3 vertices, or non-positive frame dims) ? an EXACT no-op (single-
20     court /
21     unconfigured footage), matching ``rally_gate``'s do-no-harm discipline.
22
23     The polygon is the court FLOOR in normalised 0-1 image coords (same convention as
24     ``FusionConfig.court_polygon``). ``height_pad`` is the MVP pseudo-3D play-volume (§4.3):
25     a point up
26     to ``height_pad`` (normalised) ABOVE the floor ? i.e. higher in frame, smaller y ? is
27     still
28     admitted, so a high clear over the prominent court is not clipped.
29
30     """
31     from __future__ import annotations
```

```

22     from dataclasses import replace
23     from typing import Any, List, Optional, Sequence, Tuple
24
25     from backend.pipeline.detectors.fusion_features import point_in_polygon
26
27     Polygon = Sequence[Tuple[float, float]]
28
29
30     def admitted(nx: float, ny: float, polygon: Polygon, height_pad: float = 0.0) -> bool:
31         """Is the normalised point inside the court floor polygon, OR within ``height_pad``
above it?
32
33         The pseudo-3D air column: shifting the point DOWN by ``height_pad`` (toward the
floor) and
34         finding it inside means the original sits within ``height_pad`` above the floor ? a
high shuttle
35         over the prominent court, admit it."""
36         if point_in_polygon((nx, ny), polygon):
37             return True
38         return height_pad > 0.0 and point_in_polygon((nx, ny + height_pad), polygon)
39
40
41     def in_court_fraction(interval: Tuple[float, float], points: List[Any], polygon:
Polygon,
42                             fps: float, frame_width: float, frame_height: float,
43                             height_pad: float = 0.0) -> float:
44         """Fraction of a candidate rally's VISIBLE shuttle points that lie inside the
prominent court.
45
46         ``interval`` is ``(start_s, end_s)``; points carry ``.frame``. Returns 1.0 when
there are no
47         visible points in the interval (a window with no shuttle is not an out-of-court
false positive ?
48         leave that judgement to the windower)."""
49         if not polygon or len(polygon) < 3 or frame_width <= 0 or frame_height <= 0 or fps
<= 0:
50             return 1.0
51         fs, fe = interval[0] * fps, interval[1] * fps
52         n = inside = 0
53         for p in points:
54             if getattr(p, "visible", False) and fs <= p.frame <= fe:
55                 n += 1
56                 if admitted(p.x / frame_width, p.y / frame_height, polygon, height_pad):
57                     inside += 1
58         return 1.0 if n == 0 else inside / n
59
60
61     def gate_intervals_to_court(intervals: List[Tuple[float, float]], points: List[Any],
62                                 polygon: Optional[Polygon], fps: float,
63                                 frame_width: float, frame_height: float,
64                                 min_in_court_frac: float = 0.5, height_pad: float = 0.0
65                                 ) -> List[Tuple[float, float]]:
66         """Drop candidate rallies whose shuttle is MAJORITY out-of-court ? the window-level
gate.
67
68         Point-level gating (:func:`gate_points_to_court`) is leaky: a background-court rally
whose shuttle
69         occasionally drifts into the prominent court keeps a few in-court points, enough to
hold a (sparse)
70         window (measured: GX010142 rally #1 is 90% adjacent-court yet a 10% central tail
survives a hard
71         point gate). This drops an interval whose in-court point fraction is below
``min_in_court_frac``,
72         so a rally that is *mostly* in the adjacent court is removed wholesale. No-op
(returns all

```

```

73         intervals) when ``polygon`` is None/degenerate. Subtractive-only."""
74         if not polygon or len(polygon) < 3 or frame_width <= 0 or frame_height <= 0 or fps
<= 0:
75             return intervals
76         return [iv for iv in intervals
77                 if in_court_fraction(iv, points, polygon, fps, frame_width, frame_height,
height_pad)
78                     >= min_in_court_frac]
79
80
81     def gate_points_to_court(points: List[Any], polygon: Optional[Polygon],
82                             frame_width: float, frame_height: float,
83                             height_pad: float = 0.0) -> List[Any]:
84         """Return ``points`` with visibility CLEARED for any visible shuttle outside the
prominent-court
85         polygon (expanded upward by ``height_pad``).
86
87         No-op (returns the input list unchanged) when ``polygon`` is None/degenerate or the
frame dims
88         are non-positive. Otherwise each gated point is a NEW ``TrajectoryPoint`` (the input
list and
89         its points are never mutated)."""
90         if not polygon or len(polygon) < 3 or frame_width <= 0 or frame_height <= 0:
91             return points
92         out: List[Any] = []
93         for p in points:
94             if getattr(p, "visible", False):
95                 nx, ny = p.x / frame_width, p.y / frame_height
96                 if not admitted(nx, ny, polygon, height_pad):
97                     p = replace(p, visible=False)
98             out.append(p)
99         return out
100

```

8. user (2026-06-19T16:59:21.666Z)

```

1     """Rally-start gate: suppress between-point false fires using shuttle net-crossings.
2
3     WASB (shuttle-only) over-detects: when a player flicks/tosses the shuttle back to
4     the opponent for service after losing a point, that single trajectory looks like a
5     rally. The discriminator is structural, not duration-based: a real rally has the
6     shuttle crossing the net MULTIPLE times (each shot lands on the other side); a
7     single service feed crosses ~once.
8
9     This module is camera-agnostic and needs NO new model ? it works on the trajectory
10    we already produce (Frame,Visibility,X,Y). It estimates the play axis (the image
11    axis with the larger shuttle spread) and the net position (median shuttle coord on
12    that axis ? the shuttle clusters at the two ends and crosses the net region often,
13    so the median sits near the net), then counts sign changes of (coord - net) across
14    the visible points inside a candidate window, with a deadband to ignore jitter.
15
16    Gate A in the over-fire fix spectrum (B = player-stance state machine, future).
17    Pure functions only ? no I/O, unit-tested; intended as a post-filter on the
18    windows from trajectory_to_action_windows, or to fold into it later.
19    """
20    from __future__ import annotations
21
22    from dataclasses import dataclass
23    from typing import List, Optional, Sequence, Tuple
24
25    Interval = Tuple[float, float]
26
27
28    @dataclass
29    class GatedWindow:

```

```

30     start: float
31     end: float
32     crossings: int
33     visible_points: int
34     kept: bool
35
36
37 def _spread(vals: Sequence[float]) -> float:
38     """Robust spread = p95 - p5 (ignores a few outlier detections)."""
39     if len(vals) < 2:
40         return 0.0
41     s = sorted(vals)
42     lo = s[max(0, int(0.05 * (len(s) - 1)))]
43     hi = s[min(len(s) - 1, int(0.95 * (len(s) - 1)))]
44     return hi - lo
45
46
47 def _median(vals: Sequence[float]) -> float:
48     if not vals:
49         return 0.0
50     s = sorted(vals)
51     n = len(s)
52     return s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2])
53
54
55 def estimate_play_axis(points) -> Tuple[str, float, float]:
56     """Return (axis 'x'/'y', net_value, span) from visible points.
57
58     Play axis = the image axis along which the shuttle travels most (larger spread).
59     For a baseline camera that's Y (players top/bottom); for a side camera, X.
60     """
61     xs = [p.x for p in points if p.visible]
62     ys = [p.y for p in points if p.visible]
63     sx, sy = _spread(xs), _spread(ys)
64     if sx >= sy:
65         return "x", _median(xs), sx
66     return "y", _median(ys), sy
67
68
69 def count_net_crossings(window_points, axis: str, net: float, deadband: float) -> int:
70     """Count sign changes of (coord - net) across visible points, ignoring the
71     deadband zone near the net (so jitter at the net doesn't inflate the count)."""
72     crossings = 0
73     last_side = None
74     for p in window_points:
75         if not p.visible:
76             continue
77         v = (p.x if axis == "x" else p.y) - net
78         if abs(v) < deadband:
79             continue
80         side = v > 0
81         if last_side is not None and side != last_side:
82             crossings += 1
83         last_side = side
84     return crossings
85
86
87 def _points_in(points, fps: float, start: float, end: float):
88     return [p for p in points if start * fps <= p.frame <= end * fps]
89
90
91 def gate_windows(points, windows: List[Interval], fps: float,
92                 min_crossings: int = 2, deadband_frac: float = 0.06,
93                 estimate: Optional[Tuple[str, float, float]] = None) ->
List[GatedWindow]:

```

```

94     """Annotate each (start,end) window with its shuttle net-crossing count and a
95     keep decision (crossings >= min_crossings). Deadband is a fraction of the play
96     axis span. Net/axis are estimated once over the whole trajectory (stable).
97
98     `estimate` lets a caller scoring MANY windows off one trajectory pass a precomputed
99     `(axis, net, span)` (from `estimate_play_axis`) so the O(P log P) estimate isn't
redone
100     per window ? same result, avoids the per-window re-estimation."""
101     axis, net, span = estimate if estimate is not None else estimate_play_axis(points)
102     deadband = deadband_frac * span
103     out: List[GatedWindow] = []
104     for (s, e) in windows:
105         wp = _points_in(points, fps, s, e)
106         c = count_net_crossings(wp, axis, net, deadband)
107         vis = sum(1 for p in wp if p.visible)
108         out.append(GatedWindow(start=s, end=e, crossings=c, visible_points=vis,
109                               kept=c >= min_crossings))
110     return out
111
112
113     def apply_gate(points, windows: List[Interval], fps: float,
114                   min_crossings: int = 2, deadband_frac: float = 0.06) -> List[Interval]:
115         """Convenience: return only the windows that pass the crossing gate."""
116         return [(g.start, g.end) for g in
117                 gate_windows(points, windows, fps, min_crossings, deadband_frac) if g.kept]
118

```

9. assistant (2026-06-19T16:59:24.435Z)

Now let me look at the design doc for the prominent-court detection and the multicourt analysis, and find the ghost/stuck-pixel handling.

10. user (2026-06-19T16:59:25.083Z)

docs\PROMINENT_COURT_DETECTION.md

11. user (2026-06-19T16:59:26.029Z)

docs\MULTI_SIGNAL_FUSION_PLAN.md

12. user (2026-06-19T16:59:26.836Z)

Found 2 files
tests\test_run_telemetry.py
docs\mockups\ui_alpha_mockup.html

13. assistant (2026-06-19T16:59:30.403Z)

The ghost / stuck-pixel concept isn't documented yet ? it lives in the prompt's context. Let me read the prominent-court design doc, the fusion features (point_in_polygon), and look for the cached detector candidate data and any existing tracker / ghost detection logic.

14. user (2026-06-19T16:59:31.314Z)

```

1     # Prominent-Court Detection ? kill background-court false positives (multicourt)
2
3     > **Status:** `DRAFT` ? design + experiment **owner-gated for any default flip** .
**Owner:** Avi . **Created:** 2026-06-19 . **Last updated:** 2026-06-19
4     > **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/` when
DONE)
5     > **Tracking anchors:** $8 progress tracker is the source of truth; pointer in memory
`current-state`.
6     > **Relation to existing docs:** attacks gap **G2** ("precision ? local over-fires,
worst on multicourt") from [`RALLY_DETECTION_GAP_CLOSURE.md`](RALLY_DETECTION_GAP_CLOSURE.md);

```

it is the **shuttle-side** counterpart of the person-side **Gate B / court mask** in `[`MULTI_SIGNAL_FUSION_PLAN.md`](MULTI_SIGNAL_FUSION_PLAN.md)`. Complements the held-shuttle **shuttle_player_colocation** (G3) ? court-localization (G2) and held-shuttle (G3) are independent precision levers.

7 > **Honesty note:** claims are tagged ``[verified]`` (checked vs code/measured) / ``[design]`` (proposed). Every signal lands **flag-gated**, **default-OFF**, promoted only on measured lift per `[`EXPERIMENT_HARNESS.md`](EXPERIMENT_HARNESS.md)` + `[[decision-rigor-norm]]`.

8

9 ---

10

11 ## 0. TL;DR

12 In **multicourt** footage the camera sees several courts. The shuttle detector (WASB) tracks **any** shuttle in frame, so a rally on a **background / adjacent court** becomes a predicted rally on the **main match** ? a false positive. The fix: identify the **prominent court** (the foreground court that occupies the majority of the frame, where the match being filmed is played) and **keep only rallies whose shuttle lives inside it**. Mechanically this re-defines the shuttle signal from ``(x, y, visible)`` to ``(x, y, visible-AND-inside-the-prominent-court)`` **before** windowing ? so a background-court shuttle reads as "not there" and never forms a rally. A flat 2D floor polygon is the MVP; a **pseudo-3D play-volume** (court floor + the air column above it) is the refinement so a high clear over the prominent court isn't wrongly rejected. Lands as a **default-OFF experiment**; the flip is gated on a Launch Report (`[[launch-report-convention]]`).

13

14 ## 1. Problem & motivating example ``[verified]``

15 - **Measured gap (G2):** local over-fires ~2x vs Gemini, and the over-fire is **worst** on multicourt ? even among long (>7s) rallies, multicourt over-fires **1.58x** (background-court games are **real** long rallies, just on the wrong court). See ``RALLY_DETECTION_GAP_CLOSURE.md`` §3 + the P7 long-rally lens.

16 - **Concrete anchor** ? GX010142 rally #1 (2026-06-19). **Owner observation:** "the first rally is just a shuttle flying in the non-prominent court? while the prominent-court players have not started." Confirmed from the cached WASB trajectory (``output/wasb/GX010142_1080p__wasb.csv``): rally #1 (``0.72?3.65s``, 2.94s) has a shuttle centroid at **meanX ? 1616** on a 1920-wide frame (784% to the right) ? spatially apart from the central cluster where most rallies live (meanX ? 1000?1300). Several other suspicious rallies (#3, #9, #12, #16) cluster in the same far-right band ? consistent with a **right-side adjacent court**.

17 - **Why a 1D threshold is not enough** ``[verified]``: the courts do not separate cleanly on X or Y alone (rally Y-centroids overlap 250?620 px across courts; the far-right band mixes with foreground play). The courts are **2D regions** under perspective ? so we need the court **geometry**, not a coordinate cutoff. This is the owner's "pseudo-3D / court model" intuition, validated by the data.

18

19 ## 2. Goal & non-goals

20 - **Goal:** on multicourt clips, drop rallies whose shuttle is outside the prominent court, **without losing real prominent-court rallies** ? measured per-video, no regression on single-court clips (where it must be a no-op or a do-no-harm).

21 - **Non-goals (v1):** multi-camera; tracking players across courts; full court-line homography for scoring; anything biometric. Single-court footage is explicitly a **no-op** (one court ? everything is "prominent").

22

23 ## 3. Decisions locked

#	Decision	Rationale
1	D1	The prominent court is a 2D region (polygon) in image space, refined to a pseudo-3D play volume 1D thresholds don't separate courts under perspective (§1)
2	D2	Gate the shuttle trajectory by the region before windowing (re-define visibility) Stops a background rally at the source ? no window ever forms there. Reuses <code>`point_in_polygon`</code>
3	D3	Default-OFF , flag-gated; single-court = no-op Zero-regression rollout (<code>[[weak-signals-retained]]</code>); no behaviour change unless a polygon is supplied + the flag set
4	D4	Reuse the existing court_polygon config + <code>`point_in_polygon`</code> ; do NOT invent a new geometry stack <code>`FusionConfig.court_polygon`</code> + <code>`fusion_features.point_in_polygon`</code> already exist <code>`[verified]`</code>
5	D5	The prominent court can be supplied (config) OR auto-derived ; auto-derivation is its own measured step Config polygon is the cheapest first experiment; auto is the product


```

path |
31
32     ## 4. Design
33
34     ### 4.1 The signal change (the core experiment)
35     Today the windower consumes the raw WASB trajectory: a per-frame `(Frame, Visibility, X,
Y)`. The change is a pre-windowing filter:
36
37     ```
38     for each frame f: if shuttle_visible(f) and NOT inside_prominent_court(X_f, Y_f): mark
f as NOT visible
39     ```
40
41     A background-court shuttle is then indistinguishable from "no shuttle" ? the velocity
windower never opens a rally there. This is purely subtractive on the shuttle stream (it can
only remove visibility, never add it), so on a correctly-drawn prominent court it cannot
manufacture rallies ? only suppress out-of-court ones. (Mirror of `rally_gate`'s "do-no-harm"
discipline.)
42
43     ### 4.2 Identifying the prominent court (four routes, cheapest first)
44     1. Configured polygon (MVP, `[design]?buildable now`): a normalized `[[x,y],?]`
court polygon per camera/clip, exactly the existing `FusionConfig.court_polygon`. Draw it once
from the dense shuttle/player cluster. Good enough to validate the whole concept on GX010142 +
the multicourt golden clips.
45     2. Player-location auto-derivation (`[design]`): run
`PersonDetector.detections_per_frame` `[verified]` exists, take the largest, most-central,
most-persistent player cluster (the foreground players are bigger boxes lower in frame), and
fit the prominent-court polygon to where those players' feet live. "Prominent = where the
match's players are." This is why the owner asked to extract person features ? players
anchor the court.
46     3. Shuttle-density auto-derivation (`[design]`): the prominent court is the densest
sustained shuttle-activity region; cluster the visible trajectory and take the dominant mode.
Person-free fallback.
47     4. Court-line detection (`[design]`, heaviest): Hough/segmentation court lines ? the
largest court quad. Most accurate, most work; deferred.
48
49     v1 ships route 1 (config) + measures; route 2 (player-anchored) is the product upgrade.
50
51     ### 4.3 Pseudo-3D play volume (the refinement)
52     A flat floor polygon rejects a shuttle at the top of a high clear over the prominent
court (it projects above the floor quad). The pseudo-3D model treats the prominent court as a
floor quad + a vertical air column: accept a shuttle if its image point lies within the
floor polygon expanded upward by a height envelope (the max plausible shuttle height
projected through the camera). Practically: the floor quad plus an upward `pad` (perspective-
aware ? larger near the net/far baseline). Start with a generous fixed upward pad (cheap, MVP);
upgrade to a per-camera height envelope only if measurement shows real rallies being clipped.
This keeps the floor tight (rejects adjacent-court floor play) while not clipping legitimate
high shuttles.
53
54     ### 4.4 Reuse map `[verified]`
55     | Need | Existing code |
56     |---|---|
57     | point-in-region test |
`backend/pipeline/detectors/fusion_features.py::point_in_polygon` |
58     | normalized court polygon config |
`backend/config/models.py::FusionConfig.court_polygon` (today: person/Gate-B only) |
59     | player boxes per frame |
`backend/pipeline/detectors/person_detector.py::PersonDetector.detections_per_frame` |
60     | trajectory parse + windowing |
`backend/pipeline/detectors/tracknet_runner.parse_trajectory_csv`, `backend/eval/windowing.py` |
61     | per-video scoring | `backend/eval/rally_seg_eval.py` (+ the `--stratify` multicourt
split, #220) |
62
63     New code is small: a `prominent_court_gate(points, polygon, height_pad)` that filters
trajectory points, threaded into the windowing input behind a default-OFF flag.

```

64

65 ### 4.5 Extension ? deterministic rally-END via shuttle-floor contact (owner note, 2026-06-19) `[design]`

66 The same court geometry that localizes the prominent court also unlocks a

67 **deterministic rally-END signal** ? the cheapest, most certain way to know a point is over:

68 **the shuttle hit the floor.** This attacks **G3** (rally-END over-extension), where today's shuttle-velocity windowing runs the window long because "shuttle still moving ? rally over."

69 - **Introduce two geometric primitives:**

70 - **The white court lines.** A badminton court is bounded by **white, straight lines forming a rectangle** (with the perspective trapezoid in image space). Detect them ? Hough lines / line-segment detection filtered to white, long, court-consistent segments ? to recover the

71 **court quad** (which also *is* the prominent-court floor polygon of §4.2 route 4; one detector serves both).

72 - **The floor plane.** The court lines define the **floor** (the plane the players stand on). The shuttle in flight lives *above* this plane; a shuttle *at* the plane (within the trapezoid, or just outside it) is **on the floor**.

73 - **The rally-END rule (deterministic).** a rally ends when the shuttle makes **floor contact** ? its trajectory descends to and **stops/rests at the floor level** (a near-zero-velocity terminus at the court-line plane), OR a shuttle that goes to the floor **outside the prominent court** = an out-of-bounds landing (also point-over). Unlike velocity heuristics, a shuttle resting on the floor is **unambiguous** ? the point is over.

74 - **Why it's strong + cheap:** no new model, no person stack ? it reuses the WASB trajectory (which already gives the shuttle's descending `(x, y)` + the `Visibility` drop when it settles) plus the court-line geometry. It is the **shuttle-side** rally-END signal, complementary to the **player-state** rally-END (player kinematics / held-shuttle) in `RALLY\_DETECTION\_GAP\_CLOSURE.md` §4 Lever A ? fuse both for robustness (a shuttle can settle off-camera; a player can pick up early).

75 - **Subtlety to measure:** distinguish a true floor landing (point over) from a shuttle that **dips low but is played back up** (a tight net shot / low retrieve). The court-line plane + a brief **dwell** at floor level (the shuttle rests, doesn't re-accelerate up) is the discriminator; tune the dwell window on golden clips.

76 This is tracked as **C8** below and cross-links gap-closure **G3**. It shares the court-line detector with the prominent-court polygon (§4.2 route 4), so the two land together once court-line detection exists.

77 ## 5. Experiment plan (measurement)

78 Land the gate flag-gated, then measure on the **cached trajectories** (CPU, no GPU/re-detect):

79 1. **GX010142 sanity:** draw a prominent-court polygon; confirm **rally #1 disappears** and the central rallies survive. The litmus the whole idea is born from.

80 2. **Multicourt golden** (`mahadevpura\_2`, GX010128, Badminton_BXH_2, Boxhill_Doubles`): ? rally-count, and once those clips are golden-labelled, ? precision / ? recall vs golden via `rally_seg_eval --stratify` (single vs multicourt). The win condition: multicourt precision up, **recall flat** (we drop FPs, not real rallies).

81 3. **Single-court no-op:** on single-court clips with no polygon (or a full-frame polygon) the gate is a verified no-op ? assert zero change.

82 4. **Dual-eval discipline** ([[eval-dual-set-regression]]): no per-video regression on either eval set.

83 Promotion to default-ON is a **separate, owner-gated** decision with a Launch Report (both rulers + the over-seg guard), never bundled with the experiment PR.

84 ## 6. Risks & limits

85 - **Polygon is camera-dependent.** A hand-drawn polygon doesn't generalize across setups ? route 2/3 (auto-derivation) is the real product path; v1 config-polygon is for *validation*.

86 - **Shuttle height** (§4.3): too-tight a floor clips high clears; the pseudo-3D pad mitigates, measurement tunes it.

87 - **The prominent court can change** (camera re-aim mid-clip): out of scope v1 (assume static per clip).

88 - **Not a recall fix.** This only removes out-of-court FPs; it does nothing for missed in-court rallies (G1) ? and must be proven not to *cost* recall.

89 ## 7. Open questions (owner / data)

```

93      1. Auto-derivation anchor: **player-location** (route 2) vs **shuttle-density** (route
94      3) as the default once validated? *(decides: data, post-experiment)*
95      2. Height-envelope: fixed upward pad vs per-camera perspective model ? is the fixed pad
96      enough? *(decides: measurement)*
97      3. Do we expose the prominent court as a **one-time per-camera setup** (draw-once UI) or
98      always auto? *(decides: owner/product)*
99
100     ## 8. Deliverables & progress tracker ? source of truth
101     Legend: ? Todo . ? In progress . ? Done . ? Gated. One small PR per row, off
102     `origin/master`, no stacking.
103
104     | ID | Deliverable | Depends on | Gated? | Status | PR |
105     |----|-----|-----|-----|-----|----|
106     | C0 | This design doc | ? | No | ? | this PR |
107     | C1 | `prominent_court_gate(points, polygon, height_pad)` + unit tests (pure, point-in-
108     polygon) | ? | No (default-OFF) | ? | ? |
109     | C2 | Thread the gate into the windowing input behind a default-OFF flag
110     (`court_polygon` + `gate_shuttle_to_court`) | C1 | No (default-OFF) | ? | ? |
111     | C3 | GX010142 litmus: rally #1 dropped, central rallies kept (measured) | C2 | No | ?
112     | ? |
113     | C4 | Extract person features (`detections_per_frame`) on GX010142 ? player-cluster =
114     prominent court; measure `shuttle_player_colocation` too | ? | No | ? | ? |
115     | C5 | Multicourt-golden measurement (? count now; ? precision/recall vs golden once
116     labelled) | C2, labels | ? owner labels | ? | ? |
117     | C6 | Auto-derive the polygon (route 2 player-anchored) | C4 | ? owner | ? | ? |
118     | C7 | Launch decision (default-ON) with a Launch Report | C3,C5 | ? owner | ? | ? |
119     | C8 | **Court-line / floor detection** ? (a) auto court polygon, (b) **deterministic
120     shuttle-floor-contact rally-END** ($4.5, targets G3) | ? | ? owner | ? | ? |
121
122     ## 9. Definition of done
123     - [ ] The gate exists, is **default-OFF**, unit-tested, and a verified no-op on single-
124     court / no-polygon.
125     - [ ] GX010142 rally #1 (and the far-right adjacent-court band) is dropped while the
126     central rallies survive `[measured]`.
127     - [ ] On the multicourt golden clips, multicourt **precision improves with recall flat**
128     vs golden labels (no per-video regression).
129     - [ ] A winning configuration is flipped default-ON only via a Launch Report ? flip
130     `Status ? DONE`, archive.
131
132     ## 10. References
133     **Code `[verified]`:** `fusion_features.py` (`point_in_polygon`, `person_in_court`),
134     `person_detector.py` (`detections_per_frame`), `config/models.py`
135     (`FusionConfig.court_polygon`, `eval/windowing.py`, `eval/rally_seg_eval.py` (`--stratify`)).
136     **Docs:** [`RALLY_DETECTION_GAP_CLOSURE.md`](RALLY_DETECTION_GAP_CLOSURE.md) (G2),
137     [`MULTI_SIGNAL_FUSION_PLAN.md`](MULTI_SIGNAL_FUSION_PLAN.md) (Gate B / court mask),
138     [`EXPERIMENT_HARNESS.md`](EXPERIMENT_HARNESS.md),
139     [`QUALITY_ITERATIONS.md`](QUALITY_ITERATIONS.md).
140
141     ---
142
143     ### Changelog
144     - 2026-06-19 ? Created from the owner's GX010142 rally-#1 (background-court FP)
145     observation + the measured multicourt over-fire (G2). Validated the 2D/pseudo-3D need from the
146     cached trajectory; mapped the reuse onto the existing `court_polygon` + `point_in_polygon`
147     machinery.
148     - 2026-06-19 ? Added $4.5 (owner note): a **deterministic rally-END signal via shuttle-
149     floor contact** ? detect the white court lines ? the floor plane ? a shuttle resting on the
150     floor (or landing out-of-court) ends the rally (targets G3). Shares the court-line detector with
151     the prominent-court polygon; tracked as C8.
152
153

```

15. user (2026-06-19T16:59:33.016Z)

Found 25 files

backend\eval\rally_seg_eval.py

```

backend\pipeline\segmenters\gemini.py
backend\providers\gemini.py
backend\main.py
backend\infrastructure\database.py
backend\eval\golden_real_fixtures.py
backend\eval\golden_fixtures.py
backend\eval\experiment.py
backend\eval\ablation.py
backend\pipeline\stitcher\compiler.py
backend\pipeline\detectors\tracknet_runner.py
backend\eval\ablation_pdf.py
backend\pipeline\segmenters\trajectory_hybrid.py
backend\eval\windowing.py
backend\pipeline\detectors\wasb_runner.py
backend\pipeline\detectors\base.py
backend\eval\serve_contrast.py
backend\pipeline\personalization\serving.py
backend\eval\per_user_regression.py
backend\eval\per_user_eval.py
backend\utils\run_telemetry.py
backend\infrastructure\execution_policy.py
backend\eval\score_calibration.py
backend\pipeline\detectors\person_detector.py
backend\eval\classifier.py

```

16. assistant (2026-06-19T16:59:36.775Z)

Now let me look at the WASB runner and tracker to understand how the single-target track is built and where stuck-pixel ghosts come from. Let me search for the tracking/stuck-pixel handling specifically.

17. user (2026-06-19T16:59:39.284Z)

```

backend\pipeline\detectors\wasb_runner.py:20:# parse_trajectory_csv /
trajectory_to_action_windows are @staticmethods on TrackNetRunner.
backend\pipeline\detectors\wasb_runner.py:190:         stale = [staged_video] if
self.cfg.stream_video else [frames_dir, staged_video]
backend\pipeline\detectors\wasb_runner.py:194:         self._wsl_rm_rf(stale)
backend\pipeline\detectors\wasb_runner.py:198:     parse_trajectory_csv =
staticmethod(TrackNetRunner.parse_trajectory_csv)
backend\pipeline\detectors\wasb_runner.py:199:     trajectory_to_action_windows =
staticmethod(TrackNetRunner.trajectory_to_action_windows)

```

18. user (2026-06-19T16:59:40.327Z)

```

1      """TrackNet shuttle-trajectory runner (WSL2 subprocess adapter).
2
3      This bridges the Windows-side indexer to a TensorFlow TrackNetV4 install living
4      inside WSL2 (ADR-001 in docs/DECISIONS.md). The flow is:
5
6          1. Translate the Windows video path to its WSL-visible form (and optionally
7             stage it into the WSL filesystem to avoid slow /mnt/c per-frame reads).
8          2. Invoke ``src/predict.py`` inside the TrackNetV4 conda env via ``wsl``.
9          3. Read back the per-frame ``Frame,Visibility,X,Y`` CSV it writes.
10         4. Convert that trajectory into high-motion "action windows" ? the same
11            candidate-rally shape HybridYoloSegmenter produces ? so the rest of the
12            pipeline (AI handoff, DB population) is unchanged.
13
14      Nothing here imports TensorFlow; all model code stays in WSL.
15      """
16
17      import os
18      import csv
19      import shlex

```

```

20     import logging
21     import subprocess
22     from dataclasses import dataclass
23     from typing import List, Dict, Any, Optional, Tuple
24
25     from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
wsl_tilde_quote
26
27     logger = logging.getLogger(__name__)
28
29
30     @dataclass
31     class TrackNetConfig:
32         """Resolved settings for a TrackNet WSL run (built from config.json ->
indexing.tracknet)."""
33         distro: str = "Ubuntu"
34         repo_dir: str = "~/models/TrackNetV4"          # WSL path to the cloned repo
35         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36         conda_env: str = "TrackNetV4"
37         weights_path: str = ""                        # WSL path to the .h5/.keras weights
(REQUIRED)
38         queue_length: int = 5
39         stage_in_wsl: bool = True                    # copy video into WSL fs first
(perf)
40         wsl_stage_dir: str = "~/clips"                # where staged videos/outputs live
in WSL
41         timeout_sec: int = 1800                      # 30 min; kills a hung call (0 = no
timeout)
42         keep_frames: bool = False                    # retain staged video after success
(debug only)
43         min_free_gb: float = 0.0                     # warn if WSL free space below this
(0 = off)
44
45     @classmethod
46     def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "TrackNetConfig":
47         tn = (idx_cfg or {}).get("tracknet", {}) or {}
48         return cls(
49             distro=tn.get("wsl_distro", "Ubuntu"),
50             repo_dir=tn.get("repo_dir", "~/models/TrackNetV4"),
51             conda_sh=tn.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
52             conda_env=tn.get("conda_env", "TrackNetV4"),
53             weights_path=tn.get("weights_path", ""),
54             queue_length=int(tn.get("queue_length", 5)),
55             stage_in_wsl=bool(tn.get("stage_in_wsl", True)),
56             wsl_stage_dir=tn.get("wsl_stage_dir", "~/clips"),
57             timeout_sec=int(tn.get("timeout_sec", 1800)),
58             keep_frames=bool(tn.get("keep_frames", False)),
59             min_free_gb=float(tn.get("min_free_gb", 0.0)),
60         )
61
62
63     def to_wsl_mnt_path(win_path: str) -> str:
64         """Translate a Windows path (C:\\Users\\x) to its WSL /mnt form (/mnt/c/Users/x).
65
66         Already-POSIX paths (starting with / or ~) are returned unchanged so the
67         same helper is safe to call on either kind of path.
68         """
69         p = str(win_path)
70         if p.startswith("/") or p.startswith("~"):
71             return p
72         p = p.replace("\\", "/")
73         if len(p) >= 2 and p[1] == ":":
74             drive = p[0].lower()
75             return f"/mnt/{drive}{p[2:]}"
76         return p

```

```

77
78
79 @dataclass
80 class TrajectoryPoint:
81     frame: int
82     visible: bool
83     x: float
84     y: float
85
86
87 class TrackNetRunner(WslCommandMixin, DetectorRunner):
88     """Runs TrackNet predict.py in WSL and turns the output CSV into action windows.
89
90     Implements the common DetectorRunner contract so a future Linux-native or
91     alternative trajectory runner can be substituted without touching the hybrids.
92     """
93
94     def __init__(self, cfg: TrackNetConfig):
95         self.cfg = cfg
96
97     def healthcheck(self) -> Tuple[bool, str]:
98         """Verify the WSL env is usable: conda env exists, TF imports, GPU visible.
99
100         Returns (ok, message). Cheap to call before a long run.
101         """
102         cmd = (
103             f"conda activate {self.cfg.conda_env} && "
104             f"python -c \"import tensorflow as tf; "
105             f"print('TF', tf.__version__, 'GPUs', "
106             f"len(tf.config.list_physical_devices('GPU')))\\""
107         )
108         try:
109             res = self._wsl_bash(cmd)
110         except FileNotFoundError:
111             return False, "`wsl` not found ? is this running on Windows with WSL2
112             installed?"
113         except subprocess.TimeoutExpired:
114             return False, "WSL healthcheck timed out."
115         out = (res.stdout or "").strip()
116         if res.returncode != 0:
117             return False, f"WSL env check failed: {(res.stderr or out).strip()}"
118         return True, out
119
120 # ----- inference
121
122 def run_predict(self, video_win_path: str, output_win_dir: str,
123                 video_id: Optional[str] = None) -> Optional[str]:
124     """Run predict.py on a video. Returns the Windows path to the produced CSV, or
125     None.
126
127     ``output_win_dir`` is a Windows directory (under the repo's output/) that
128     WSL writes into via its /mnt view, so the Windows process can read the CSV
129     back with no copy. ``video_id`` (file MD5) namespaces the staged video ? and
130     therefore predict.py's ``<stem>_predict.csv`` output ? so two videos that share
131     a filename cannot collide on the output CSV.
132     """
133     if not self.cfg.weights_path:
134         logger.error(
135             "TrackNet weights_path is not set. Set indexing.tracknet.weights_path "
136             "in config.json to the WSL path of your trained TrackNetV4 weights."
137         )
138     return None
139
140 # Resolve relative dirs BEFORE the /mnt translation ? to_wsl_mnt_path passes
141 # relative paths through unchanged, so WSL would resolve them against the

```

```

139     # predict.py cwd instead of the repo (same bug class as wasb_runner).
140     output_win_dir = os.path.abspath(output_win_dir)
141     os.makedirs(output_win_dir, exist_ok=True)
142     # Normalize separators so basename/splitext work when tests (or collector)
143     # pass Windows-style paths on a Linux host.
144     video_win_path = str(video_win_path).replace("\\", "/")
145     base = os.path.basename(video_win_path)
146     stem, ext = os.path.splitext(base)
147
148     # predict.py only accepts .avi/.mp4 and names the CSV "<stem>_predict.csv".
149     if ext.lower() not in (".mp4", ".avi"):
150         logger.error("TrackNet predict.py only supports .mp4/.avi (got %s)", ext)
151         return None
152
153     wsl_out = to_wsl_mnt_path(output_win_dir)
154     # Namespace by video_id so two same-filename videos don't collide on the CSV.
155     # predict.py derives its output name from the (staged) video stem, so staging
156     # under the namespaced name propagates into "<key>_predict.csv".
157     key = f"{stem}_{video_id[:12]}" if video_id else stem
158     expected_csv_win = os.path.join(output_win_dir, f"{key}_predict.csv")
159
160     # Resolve the video path WSL will read.
161     if self.cfg.stage_in_wsl:
162         staged = self._stage_video(video_win_path, f"{key}{ext}")
163         if staged is None:
164             return None
165         wsl_video = staged
166     else:
167         # No staging: predict.py reads /mnt directly and writes
168         "<stem>_predict.csv".
169         # We cannot rename its output, so fall back to the un-namespaced name.
170         wsl_video = to_wsl_mnt_path(video_win_path)
171         expected_csv_win = os.path.join(output_win_dir, f"{stem}_predict.csv")
172
173     if self.cfg.min_free_gb > 0:
174         free = self._wsl_free_gb(self.cfg.wsl_stage_dir)
175         if free is not None and free < self.cfg.min_free_gb:
176             logger.warning("Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f); "
177                             "consider running tools/cleanup_caches.py.",
178                             free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)
179
180     cmd = (
181         f"conda activate {self.cfg.conda_env} && "
182         f"cd {self.cfg.repo_dir}/src && "
183         f"python predict.py "
184         f"--video_path {wsl_tilde_quote(wsl_video)} "
185         f"--model_weights {wsl_tilde_quote(self.cfg.weights_path)} "
186         f"--output_dir {wsl_tilde_quote(wsl_out)} "
187         f"--queue_length {self.cfg.queue_length}"
188     )
189     logger.info("Running TrackNet inference on %s ...", base)
190     try:
191         res = self._wsl_bash(cmd)
192     except subprocess.TimeoutExpired:
193         logger.error("TrackNet inference timed out after %ss.",
194                     self.cfg.timeout_sec)
195         return None
196
197     if res.returncode != 0:
198         logger.error("TrackNet predict.py failed:\n%s", (res.stderr or
199                     res.stdout)[-2000:])
200         return None
201
202     if not os.path.exists(expected_csv_win):
203         logger.error("TrackNet finished but expected CSV not found at %s",

```

```

expected_csv_win)
201         return None
202         logger.info("TrackNet trajectory CSV: %s", expected_csv_win)
203
204         # Success ? the CSV is the durable output; drop the staged video copy (a pure,
205         # regenerable intermediate). On earlier failure we return above without
cleaning.
206         if self.cfg.stage_in_wsl and not self.cfg.keep_frames:
207             logger.info("Cache hygiene: removing staged video for %s "
208                         "(set indexing.tracknet.keep_frames=true to retain).", key)
209             self._wsl_rm_rf([wsl_video])
210             return expected_csv_win
211
212     def _stage_video(self, video_win_path: str, base: str) -> Optional[str]:
213         """Copy the input video into the WSL filesystem (avoids slow /mnt/c reads).
214
215         Returns the WSL path to the staged copy, or None on failure.
216         """
217         src_mnt = to_wsl_mnt_path(video_win_path)
218         reason = self.wsl_source_unreadable_reason(src_mnt)
219         if reason:
220             logger.error("Cannot stage video into WSL: %s", reason)
221             return None
222         dest = f"{self.cfg.wsl_stage_dir}/{base}"
223         cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}"
224         {wsl_tilde_quote(dest)} && echo OK"
225         try:
226             res = self._wsl_bash(cmd)
227         except subprocess.TimeoutExpired:
228             logger.error("Staging video into WSL timed out.")
229             return None
230         if res.returncode != 0 or "OK" not in (res.stdout or ""):
231             logger.error("Failed to stage video into WSL: %s", (res.stderr or
232             res.stdout).strip())
233             return None
234             return dest
235
236     # ----- CSV -> windows
237
238     @staticmethod
239     def parse_trajectory_csv(csv_win_path: str) -> List[TrajectoryPoint]:
240         """Parse a TrackNet predict CSV (columns: Frame,Visibility,X,Y)."""
241         points: List[TrajectoryPoint] = []
242         with open(csv_win_path, newline="") as f:
243             reader = csv.DictReader(f)
244             for row in reader:
245                 try:
246                     points.append(TrajectoryPoint(
247                         frame=int(row["Frame"]),
248                         visible=int(row["Visibility"]) == 1,
249                         x=float(row["X"]),
250                         y=float(row["Y"]),
251                     ))
252                 except (KeyError, ValueError):
253                     continue
254             points.sort(key=lambda p: p.frame)
255             return points
256
257     @staticmethod
258     def _active_frames(points: List[TrajectoryPoint], fps: float, frame_width: float,
259                        velocity_thresh: float) -> List[Tuple[int, float]]:
260         """Per-frame ``(frame_idx, velocity)`` for frames where the shuttle is in
flight.
261
262         Computes each visible point's normalised velocity from the last *visible* point;

```



```

261         an absent shuttle breaks the trajectory. A frame qualifies when its velocity
exceeds
262         ``velocity_thresh``. ``fps``/``frame_width`` are expected to be already
defaulted by
263         the caller so identical floats feed the velocity math."""
264         active = [] # (frame_idx, velocity)
265         prev: Optional[TrajectoryPoint] = None
266         for p in points:
267             if not p.visible:
268                 prev = None # break the trajectory; absent shuttle = not in flight
269                 continue
270             if prev is not None:
271                 dframes = p.frame - prev.frame
272                 if dframes > 0:
273                     dist = ((p.x - prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5
274                     velocity = (dist / frame_width) * (fps / dframes)
275                     if velocity > velocity_thresh:
276                         active.append((p.frame, velocity))
277             prev = p
278         return active
279
280     @staticmethod
281     def trajectory_to_action_windows(
282         points: List[TrajectoryPoint],
283         fps: float,
284         frame_width: float,
285         chunk_start: float = 0.0,
286         velocity_thresh: float = 0.02,
287         merge_gap: float = 2.0,
288         min_window_duration: float = 2.0,
289         chunk_index: Optional[int] = None,
290         max_window_duration: float = 0.0,
291         min_split_gap: float = 0.5,
292         window_hard_cap: bool = False,
293         window_inactivity_end: float = 0.0,
294         inactivity_min_density: float = 0.34,
295         inactivity_rally_thresh: float = 0.08,
296         window_blob_fallback: bool = False,
297         window_blob_factor: float = 4.0,
298     ) -> List[Dict[str, Any]]:
299         """Convert a shuttle trajectory into candidate rally windows.
300
301         A frame is "active" when the shuttle is visible AND its inter-frame
302         displacement (normalised by frame width, per second) exceeds
303         ``velocity_thresh`` ? i.e. the shuttle is in flight. Active frames within
304         ``merge_gap`` seconds of each other are grouped into one window; short
305         windows are dropped. Mirrors HybridYoloSegmenter's windowing so the dict
306         shape feeding the AI-handoff phase is identical.
307
308         ``max_window_duration`` (0 = OFF, the default ? behaviour unchanged): a guard
against
309         the *over-merge* failure where "shuttle moving" is conflated with "rally in
progress"
310         and a single window swallows several rallies + the dead time between them. When
311         window longer than this is recursively SPLIT at its largest internal inactivity
gap (the
312         longest stretch with no in-flight shuttle ? the most likely rally boundary),
provided
313         that gap is at least ``min_split_gap`` seconds. This is the bounded-windowing
fix from
314         ``docs/RALLY_QUALITY_RESEARCH.md`` §6 (W1); it never merges, only cuts, so
recall cannot
315         drop, and it is config-gated default-OFF until measured on the golden set.
316

```

```

317         ``window_hard_cap`` (default OFF) makes ``max_window_duration`` a TRUE cap: when
an
318         over-long window has NO internal inactivity gap to split on (a continuously-
active
319         trajectory ? the shuttle never stops moving, e.g. the real-footage
``mahadevpura-1``
320         blob where the gap-splitter alone left a single 174 s window), it is recursively
321         hard-chopped at its temporal midpoint until every piece is ? the cap. Still only
cuts
322         (never merges), so recall cannot drop; gated default-OFF until measured.
323
324         ``window_blob_fallback`` (default OFF ? the R5 last-resort blob splitter):
unlike
325         ``window_hard_cap`` (which midpoint-chops BEFORE the R4 trim, so it fires on
every gap-less
326         over-cap window and over-segments normal clips), this runs AFTER the R4
inactivity trim and
327         force-chops ONLY a genuine *blob* ? a group still longer than
``window_blob_factor`` *
328         ``max_window_duration`` once the gap-split and the principled rally-end trim
have both had
329         their chance (e.g. mahadevpura-1's ~65 s residual ? 11x a 6 s cap; a 7 s rally
is left
330         alone). The blob is midpoint-chopped down to ? the cap, those forced cuts are
logged, and
331         each resulting window is flagged ``forced_cut=True`` ? surfacing the clip as one
that needs
332         rally-STATE cues (net-crossings / two-sided motion), not a bigger cap. Only cuts
(recall
333         can't drop). Requires ``max_window_duration`` > 0; gated default-OFF until
measured.
334         ""
335         if fps <= 0:
336             fps = 30.0
337         if frame_width <= 0:
338             frame_width = 1280.0
339
340         active = TrackNetRunner._active_frames(points, fps, frame_width,
velocity_thresh)
341
342         if not active:
343             return []
344
345         max_gap_frames = int(merge_gap * fps)
346
347         def _finalize(group: List[Tuple[int, float]]) -> Dict[str, Any]:
348             vels = [v for _, v in group]
349             return {
350                 "start": group[0][0] / fps + chunk_start,
351                 "end": group[-1][0] / fps + chunk_start,
352                 "active_frame_count": len(group),
353                 "peak_velocity": max(vels),
354                 "mean_velocity": sum(vels) / len(vels),
355                 "track_id_count": 1, # single shuttle; kept for window-shape parity
356                 "chunk_index": chunk_index,
357             }
358
359         groups: List[List[Tuple[int, float]]] = []
360         group = [active[0]]
361         for af in active[1:]:
362             if af[0] - group[-1][0] <= max_gap_frames:
363                 group.append(af)
364             else:
365                 groups.append(group)
366                 group = [af]

```

```

367         groups.append(group)
368
369         if max_window_duration > 0:
370             groups = TrackNetRunner._split_overlong_groups(
371                 groups, int(max_window_duration * fps), int(min_split_gap * fps),
372                 hard_cap=window_hard_cap)
373
374         # R4 ? rally-END inactivity trim (default OFF). Applied AFTER the cap split so
each rally
375         # piece has its own post-rally drift tail removed: the velocity windowing keeps
a window
376         # "active" while the shuttle drifts/gets-picked-up above threshold, over-
extending the END
377         # (the measured gap vs golden). Trim back to the last frame whose trailing
window is still
378         # dense with motion (rally on); a sustained low-density run = rally over. Only
cuts.
379         if window_inactivity_end > 0:
380             tw = int(window_inactivity_end * fps)
381             groups = [TrackNetRunner._trim_inactive_tail(g, tw, inactivity_min_density,
382                                                         inactivity_rally_thresh) for g
in groups]
383
384         # R5 ? blob-fallback (default OFF). LAST resort: after the gap-split AND the R4
trim, any
385         # group still longer than the cap is a gap-less, rally-end-signal-less blob;
force-chop it to
386         # ? the cap at the midpoint and flag every piece, so the degenerate single-
window outcome is
387         # avoided and the clip is surfaced (logged) as needing rally-STATE cues, not a
bigger cap.
388         forced_flags = [False] * len(groups)
389         if window_blob_fallback and max_window_duration > 0:
390             groups, forced_flags = TrackNetRunner._blob_fallback_chop(
391                 groups, int(max_window_duration * fps), window_blob_factor)
392
393         windows = []
394         for g, forced in zip(groups, forced_flags):
395             w = _finalize(g)
396             if forced:
397                 w["forced_cut"] = True
398             windows.append(w)
399         return [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
400
401     @staticmethod
402     def _trim_inactive_tail(group: List[Tuple[int, float]], trail_frames: int,
403                             min_density: float, rally_thresh: float) -> List[Tuple[int,
float]]:
404         """R4: trim the post-rally drift tail. After a rally the shuttle keeps moving
*slowly*
405         (picked up / walked / knock-up) above ``velocity_thresh``, so the velocity
windowing
406         over-extends the END. A frame is "fast" (real rally motion) iff its velocity >
``rally_thresh``
407         (higher than the active threshold). The rally is "on" at frame ``i`` while its
trailing
408         ``trail_frames`` window holds >= ``min_density`` fraction of fast frames; trim
back to the
409         LAST such frame ? everything after is sustained slow drift (rally over). Only
cuts (recall
410         can't rise); a window whose tail never goes slow is untouched, and if no fast-
dense point
411         exists at all the group is kept whole (fail-safe, no over-trim). O(n) two-
pointer; the START
412         is left alone (already ~Gemini-accurate per the n=2 boundary-error finding)."""

```

```

413         if trail_frames <= 0 or len(group) < 2:
414             return group
415         frames = [f for f, _ in group]
416         fast = [1 if v > rally_thresh else 0 for _, v in group]
417         lo = 0
418         fast_sum = 0
419         last_dense = -1
420         for i in range(len(frames)):
421             fast_sum += fast[i]
422             while frames[lo] <= frames[i] - trail_frames:
423                 fast_sum -= fast[lo]
424                 lo += 1
425             win_count = i - lo + 1
426             if win_count > 0 and fast_sum / win_count >= min_density:
427                 last_dense = i
428         return group[: last_dense + 1] if last_dense >= 0 else group
429
430     @staticmethod
431     def _midpoint_split_index(group: List[Tuple[int, float]]) -> int:
432         """Index of the active frame nearest the group's temporal midpoint (used to chop
433         a
434         gap-less group into two). Shared by the W1 hard-cap fallback and the R5 blob-
435         fallback."""
436         mid = (group[0][0] + group[-1][0]) / 2.0
437         split_i = min(range(1, len(group)), key=lambda i: abs(group[i][0] - mid))
438         return split_i
439
440     @staticmethod
441     def _split_overlong_groups(groups: List[List[Tuple[int, float]]], max_win_frames:
442     int,
443     min_split_frames: int,
444     hard_cap: bool = False) -> List[List[Tuple[int, float]]]:
445         """Recursively split any active-frame group longer than ``max_win_frames`` at
446         its largest
447         internal gap (? ``min_split_frames``) ? an inactivity-aware cut at the most
448         likely rally
449         boundary. Splitting only subdivides existing groups (never merges/extends), so
450         it cannot
451         invent or drop coverage; a group with no gap ? the floor is left intact (a
452         genuinely long
453         rally) UNLESS ``hard_cap`` is set, in which case such a group is hard-chopped at
454         its
455         temporal midpoint until every piece is ? ``max_win_frames`` (makes the cap a
456         true upper
457         bound on continuously-active trajectories). Iterative worklist to bound stack
458         depth."""
459         if max_win_frames <= 0:
460             return groups
461         out: List[List[Tuple[int, float]]] = []
462         work = list(groups)
463         while work:
464             g = work.pop()
465             if len(g) < 2 or (g[-1][0] - g[0][0]) <= max_win_frames:
466                 out.append(g)
467                 continue
468             best_i, best_gap = -1, -1
469             for i in range(1, len(g)):
470                 gap = g[i][0] - g[i - 1][0]
471                 if gap > best_gap:
472                     best_gap, best_i = gap, i
473             if best_i < 0 or best_gap < min_split_frames:
474                 # Too long but no real dead-time gap. Default: keep as one (a genuine
475                 long rally).
476                 out.append(g)
477             else:
478                 # hard_cap: chop at the temporal midpoint so the cap is actually
479                 enforced.

```

```

466         if hard_cap:
467             split_i = TrackNetRunner._midpoint_split_index(g)
468             work.append(g[:split_i])
469             work.append(g[split_i:])
470         else:
471             out.append(g)
472             continue
473         work.append(g[:best_i])
474         work.append(g[best_i:])
475     out.sort(key=lambda grp: grp[0][0])
476     return out
477
478     @staticmethod
479     def _blob_fallback_chop(groups: List[List[Tuple[int, float]]], max_win_frames: int,
480                             blob_factor: float = 4.0
481                             ) -> Tuple[List[List[Tuple[int, float]]], List[bool]]:
482         """R5 blob splitter ? the gap-less, R4-survived residual. After the W1 gap-split
AND the R4
483         inactivity trim have each had their chance, a group still longer than
``blob_factor`` x
484         ``max_win_frames`` is a genuine continuously-active blob (no internal gap, no
rally-end
485         signal the principled rules could find ? the real-footage ``mahadevpura-1`` ~65
s residual ?
486         11x a 6 s cap). The factor is what keeps this SURGICAL: a normal long rally
(~1?2x the cap)
487         is left ALONE; only the degenerate blob qualifies. As a LAST RESORT the blob is
488         midpoint-chopped to ? the cap and every resulting piece is FLAGGED as a forced
cut, so the
489         degenerate single-window outcome is avoided and the clip is surfaced (logged) as
needing
490         rally-STATE cues ? net-crossings / two-sided motion ? not a bigger cap. Only
cuts (recall
491         can't drop). Returns ``(groups, forced_flags)`` aligned 1:1; ``forced_flags[i]``
is True iff
492         that piece came from a forced chop. Iterative worklist to bound stack depth."""
493         if max_win_frames <= 0:
494             return groups, [False] * len(groups)
495         blob_frames = max_win_frames * max(1.0, blob_factor)
496         out: List[List[Tuple[int, float]]] = []
497         flags: List[bool] = []
498         forced = 0
499         for g in groups:
500             if len(g) < 2 or (g[-1][0] - g[0][0]) <= blob_frames:
501                 out.append(g)
502                 flags.append(False)
503                 continue
504             pieces: List[List[Tuple[int, float]]] = []
505             work = [g]
506             while work:
507                 h = work.pop()
508                 if len(h) < 2 or (h[-1][0] - h[0][0]) <= max_win_frames:
509                     pieces.append(h)
510                     continue
511                 split_i = TrackNetRunner._midpoint_split_index(h)
512                 work.append(h[:split_i])
513                 work.append(h[split_i:])
514             pieces.sort(key=lambda grp: grp[0][0])
515             out.extend(pieces)
516             flags.extend([True] * len(pieces))
517             forced += len(pieces) - 1
518         if forced:
519             logger.warning(
520                 "R5 blob-fallback: forced %d midpoint cut(s) on continuously-active
group(s) with "

```

```

521             "no rally-end signal ? this clip needs rally-STATE cues (net-crossings /
two-sided "
522             "motion), not a bigger cap", forced)
523         return out, flags
524

```

19. assistant (2026-06-19T16:59:45.581Z)

Now I understand the windowing math deeply. Critically: `\_active\_frames` computes velocity from the **last visible point**, and breaks the trajectory on `not p.visible`. A stuck pixel at constant (x,y) has **zero displacement ? zero velocity ? never active** in the *current* single-track pipeline. This is key to understanding whether the ghost is even a windowing problem today, and whether the proposed multi-track design re-introduces it. Let me look at the windowing eval and how the proposed multi-track layer would compute motion energy. Let me also confirm the ghost data characteristics from any cached artifacts.

20. user (2026-06-19T16:59:46.589Z)

```

1      """Single-sourced candidate-windowing presets ? the eval?serve foundation fix.
2
3      **The bug this closes.** The offline eval substrate (`distill_local.build_candidates`
?
4      `fusion_golden` ? the golden feature JSONs the ablation/compare path re-scores) was
built
5      on a *recall-oriented PROPOSAL* windowing (`vt=0.005, merge_gap=1.0, min_window=0.5`):
6      deliberately permissive so a learned scorer/gate can filter the junk. But the SERVED
pipeline
7      (`TrajectoryHybridSegmenter.process_video`) windows COARSELY (`vt=0.02,
merge_gap=2.0,
8      min_window=2.0` ? the `config.indexing` defaults). A cue measured on the proposal set
is
9      measured on candidates **production never serves**, so a "win" need not transfer. This
is
10     exactly how Gate-A scored **+0.074** on the proposal set yet **?0.008** on the served
set
11     (`scratch/GATE_AB_NUMBERS_AND_IMPLICATIONS.md` vs `output/gateA_served_verify/`).
12
13     **The fix.** One module owns the two named windowings as :class:`WindowingPreset`s, and
the
14     SERVED preset is **single-sourced from the same Pydantic config defaults production
reads**
15     (:class:`backend.config.models.IndexingConfig`) ? eval and serve can no longer drift.
Helpers
16     build candidates under either preset, and :func:`served_windowing_from_config` lifts the
17     *actual* served windowing out of a live config (overrides included), so an A/B can
target the
18     operating point a given deployment truly serves.
19
20     The companion rule ? *a cue is promotable only if it does NOT regress at the SERVED
windowing* ? lives in :mod:`backend.eval.promotion` (which composes `eval_stats` +
21     `per_user_gate`). This module is the **what windowing**; that one is the **promotion
gate**.
22
23
24     Pure, offline, stdlib + config only. Nothing here changes existing eval behaviour until
a
25     caller opts in (`distill_local` re-points its module-level presets here, byte-
identically).
26     """
27     from __future__ import annotations
28
29     from dataclasses import dataclass
30     from typing import Any, Dict, List, Mapping, Tuple
31
32     from backend.config.models import IndexingConfig
33

```

```

34     Interval = Tuple[float, float]
35
36     __all__ = [
37         "WindowingPreset",
38         "SERVED",
39         "PROPOSAL",
40         "PRESETS",
41         "served_windowing_from_config",
42         "build_windows",
43         "windows_to_intervals",
44     ]
45
46
47     @dataclass(frozen=True)
48     class WindowingPreset:
49         """A named (velocity_thresh, merge_gap, min_window_duration) windowing operating
50 point.
51
52         ``velocity_thresh`` ? normalised per-second shuttle velocity above which a frame
53 counts as
54 "in flight"; ``merge_gap`` ? seconds within which active frames coalesce into one
55 window;
56
57         ``min_window_duration`` ? windows shorter than this are dropped. These are exactly
58 the three
59 knobs ``TrackNetRunner.trajectory_to_action_windows`` takes, so a preset *is* the
60 windowing.
61
62         """
63
64         name: str
65         velocity_thresh: float
66         merge_gap: float
67         min_window_duration: float
68         #: 0 = OFF (default). When > 0, windows longer than this are split at their largest
69 internal
70         #: inactivity gap ? the bounded-windowing over-merge fix (W1,
71 RALLY_QUALITY_RESEARCH.md §6).
72         max_window_duration: float = 0.0
73         #: When True, ``max_window_duration`` is a HARD cap ? an over-long window with no
74 internal
75         #: inactivity gap is chopped at its midpoint until ? the cap (continuously-active
76 fix). OFF by default.
77         hard_cap: bool = False
78         #: R4 rally-END inactivity trim (s, 0 = OFF): trim a window's post-rally slow-drift
79 tail back to
80         #: the last frame whose trailing window stays dense with FAST motion. The measured
81 rally-end fix.
82         inactivity_end: float = 0.0
83         #: velocity above which motion counts as "rally" (vs slow drift) for the R4 trim;
84 min fast-fraction.
85         inactivity_rally_thresh: float = 0.08
86         inactivity_min_density: float = 0.34
87         #: R5 blob-fallback (default OFF). After the cap split + R4 trim, midpoint-chop any
88 group still
89         #: longer than ``blob_factor`` × ``max_window_duration`` (the gap-less, R4-survived
90 blob ?
91         #: mahadevpura-1's ~11× residual) and flag it. Runs AFTER R4 (vs ``hard_cap``
92 before) and the
93         #: factor keeps it surgical (a normal long rally is left alone).
94         blob_fallback: bool = False
95         blob_factor: float = 4.0
96
97         def as_tuple(self) -> Tuple[float, float, float]:
98             """``(velocity_thresh, merge_gap, min_window_duration)`` ? the legacy tuple
99 shape the
100
101             ``trajectory_to_action_windows`` / ``distill_local`` call sites already speak.

```

```

(The
83         bounded-windowing knob ``max_window_duration`` is passed separately by
``build_windows``.)"""
84         return (self.velocity_thresh, self.merge_gap, self.min_window_duration)
85
86     def to_dict(self) -> Dict[str, Any]:
87         return {
88             "name": self.name,
89             "velocity_thresh": self.velocity_thresh,
90             "merge_gap": self.merge_gap,
91             "min_window_duration": self.min_window_duration,
92             "max_window_duration": self.max_window_duration,
93             "hard_cap": self.hard_cap,
94             "inactivity_end": self.inactivity_end,
95             "inactivity_rally_thresh": self.inactivity_rally_thresh,
96             "inactivity_min_density": self.inactivity_min_density,
97             "blob_fallback": self.blob_fallback,
98             "blob_factor": self.blob_factor,
99         }
100
101
102     def _served_preset_from_defaults() -> WindowingPreset:
103         """The SERVED windowing, lifted from the SAME Pydantic config defaults production
reads.
104
105         ``TrajectoryHybridSegmenter.process_video`` reads exactly these three:
106         - velocity: ``idx_cfg[<family>].velocity_threshold`` (default 0.02 ?
wasb/fusion/tracknet)
107         - merge_gap: ``idx_cfg.dead_time_merge_gap`` (default 2.0)
108         - min window: ``idx_cfg.min_action_window_duration`` (default 2.0)
109
110         Sourcing them from :class:`IndexingConfig`'s defaults (not a hand-typed tuple) is
the whole
111         point: change a served default in ``config/models.py`` and the eval SERVED preset
follows,
112         so the two can never silently diverge again.
113         """
114         idx = IndexingConfig() # construct with defaults ? the served operating point
115         return WindowingPreset(
116             name="served",
117             velocity_thresh=float(idx.wasb.velocity_threshold), # == fusion/tracknet
default 0.02
118             merge_gap=float(idx.dead_time_merge_gap),
119             min_window_duration=float(idx.min_action_window_duration),
120             max_window_duration=float(idx.max_window_duration), # W1 bounded-windowing
(default 0=OFF)
121             hard_cap=bool(idx.window_hard_cap), # W1 hard-cap fallback
(default OFF)
122             inactivity_end=float(idx.window_inactivity_end), # R4 rally-end trim
(default 0=OFF)
123             inactivity_rally_thresh=float(idx.inactivity_rally_thresh),
124             inactivity_min_density=float(idx.inactivity_min_density),
125             blob_fallback=bool(idx.window_blob_fallback), # R5 blob-fallback
(default OFF)
126             blob_factor=float(idx.window_blob_factor),
127         )
128
129
130     #: The COARSE windowing production actually serves (config defaults; ~0.02/2.0/2.0). A
cue is
131     #: only promotable if it holds up HERE ? the operating point real users get.
132     SERVED: WindowingPreset = _served_preset_from_defaults()
133
134     #: The recall-oriented PROPOSAL windowing the offline substrate is built on
(deliberately

```



```

135     #: permissive: propose many, let the classifier/gate filter). NOT what production serves
? a
136     #: win here must be re-confirmed on SERVED before promotion. Mirrors the historical
137     #: ``distill_local.PROPOSAL`` tuple (0.005/1.0/0.5) so the existing golden JSONs stay
valid.
138     PROPOSAL: WindowingPreset = WindowingPreset(
139         name="proposal", velocity_thresh=0.005, merge_gap=1.0, min_window_duration=0.5
140     )
141
142     #: Name ? preset, so a CLI / config string ("served" | "proposal") selects one.
143     PRESETS: Dict[str, WindowingPreset] = {SERVED.name: SERVED, PROPOSAL.name: PROPOSAL}
144
145
146     def served_windowing_from_config(config: Any, family: str = "wasb") -> WindowingPreset:
147         """The SERVED windowing a *specific* live config serves (overrides honoured).
148
149         Reproduces ``TrajectoryHybridSegmenter.process_video``'s knob reads exactly, against
an
150         arbitrary config (the typed :class:`AppConfig` or a plain dict) and segmenter
``family``
151         (``"wasb"`` / ``"fusion"`` / ``"tracknet"``). Use this when a deployment overrode a
served
152         default in ``config.json`` ? the module-level :data:`SERVED` is the *default*
operating point;
153         this is *this config's* operating point. Falls back to the served defaults for any
absent key.
154         """
155         idx = _as_mapping(config, "indexing")
156         family_cfg = _as_mapping(idx, family)
157         return WindowingPreset(
158             name=f"served:{family}",
159             velocity_thresh=float(family_cfg.get("velocity_threshold",
SERVED.velocity_thresh)),
160             merge_gap=float(idx.get("dead_time_merge_gap", SERVED.merge_gap)),
161             min_window_duration=float(idx.get("min_action_window_duration",
SERVED.min_window_duration)),
162             max_window_duration=float(idx.get("max_window_duration",
SERVED.max_window_duration)),
163             hard_cap=bool(idx.get("window_hard_cap", SERVED.hard_cap)),
164             inactivity_end=float(idx.get("window_inactivity_end", SERVED.inactivity_end)),
165             inactivity_rally_thresh=float(idx.get("inactivity_rally_thresh",
SERVED.inactivity_rally_thresh)),
166             inactivity_min_density=float(idx.get("inactivity_min_density",
SERVED.inactivity_min_density)),
167             blob_fallback=bool(idx.get("window_blob_fallback", SERVED.blob_fallback)),
168             blob_factor=float(idx.get("window_blob_factor", SERVED.blob_factor)),
169         )
170
171
172     def _as_mapping(obj: Any, key: str) -> Mapping[str, Any]:
173         """``obj[key]`` as a plain mapping, whether ``obj`` is a dict or a typed config
section
174         (``AppConfig.get`` returns nested sections as dicts). Empty mapping if absent/non-
mapping."""
175         if obj is None:
176             return {}
177         if isinstance(obj, Mapping):
178             sub = obj.get(key, {})
179         elif hasattr(obj, "get"):
180             sub = obj.get(key, {})
181         else:
182             sub = getattr(obj, key, {})
183         return sub if isinstance(sub, Mapping) else {}
184
185

```

```

186     def build_windows(points: List[Any], fps: float, frame_width: float,
187                       preset: WindowingPreset) -> List[Dict[str, Any]]:
188         """Trajectory ``points`` ? candidate windows under ``preset`` (the one windowing
call site).
189
190         A thin wrapper over ``TrackNetRunner.trajectory_to_action_windows`` that takes a
named preset
191         instead of three loose floats, so eval candidate-building and the served path
reference the
192         SAME knobs by name. Imported lazily to keep this module import-light (no detector
deps)."""
193         from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
194
195         vt, mg, mwd = preset.as_tuple()
196         return TrackNetRunner.trajectory_to_action_windows(
197             points, fps=fps, frame_width=frame_width,
198             velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd,
199             max_window_duration=preset.max_window_duration,
200             window_hard_cap=preset.hard_cap,
201             window_inactivity_end=preset.inactivity_end,
202             inactivity_rally_thresh=preset.inactivity_rally_thresh,
203             inactivity_min_density=preset.inactivity_min_density,
204             window_blob_fallback=preset.blob_fallback,
205             window_blob_factor=preset.blob_factor)
206
207
208     def windows_to_intervals(windows: List[Dict[str, Any]]) -> List[Interval]:
209         """``[{start, end, ...}]`` ? ``[(start, end)]`` ? the interval shape the scorer/gate
speak."""
210         return [(w["start"], w["end"]) for w in windows]
211

```

21. user (2026-06-19T16:59:50.128Z)

```

total 7472
drwxr-xr-x 1 avidu 197609      0 Jun 19 17:27 .
drwxr-xr-x 1 avidu 197609      0 Jun 19 22:05 ..
drwxr-xr-x 1 avidu 197609      0 Jun 15 21:58 Adarsh v Avi__bb961d0d0cff_wasb_wasbcache
-rw-r--r-- 1 avidu 197609 1635249 Jun 13 14:43 GX010125_1080p__89605e80ed01_wasb.csv
drwxr-xr-x 1 avidu 197609      0 Jun 13 14:42 GX010125_1080p__89605e80ed01_wasb_wasbcache
-rw-r--r-- 1 avidu 197609 864224 Jun 19 14:10 GX010142_1080p__c4cfa94a792f_wasb.csv
drwxr-xr-x 1 avidu 197609      0 Jun 19 14:10 GX010142_1080p__c4cfa94a792f_wasb_wasbcache
-rw-r--r-- 1 avidu 197609 774170 Jun 19 14:34 GX010143_1080p__c991e53106c8_wasb.csv
drwxr-xr-x 1 avidu 197609      0 Jun 19 14:34 GX010143_1080p__c991e53106c8_wasb_wasbcache
-rw-r--r-- 1 avidu 197609 1335313 Jun 19 16:44 GX010144_1080p__74affb36e15e_wasb.csv
drwxr-xr-x 1 avidu 197609      0 Jun 19 16:44 GX010144_1080p__74affb36e15e_wasb_wasbcache
-rw-r--r-- 1 avidu 197609 947629 Jun 19 15:32 GX010145_1080p__5a6fe8c350d4_wasb.csv
drwxr-xr-x 1 avidu 197609      0 Jun 19 15:32 GX010145_1080p__5a6fe8c350d4_wasb_wasbcache
-rw-r--r-- 1 avidu 197609 1634538 Jun 19 17:27 GX010147_1080p__79ae625eac6d_wasb.csv
drwxr-xr-x 1 avidu 197609      0 Jun 19 17:27 GX010147_1080p__79ae625eac6d_wasb_wasbcache
-rw-r--r-- 1 avidu 197609 172447 Jun 11 18:28 mahadevpura_smoke_180s__05e0e577029d_wasb.csv
drwxr-xr-x 1 avidu 197609      0 Jun 11 18:28
mahadevpura_smoke_180s__05e0e577029d_wasb_wasbcache
-rw-r--r-- 1 avidu 197609 171660 Jun 19 13:25 smoke_GX010143__1803d4c01a73_wasb.csv
drwxr-xr-x 1 avidu 197609      0 Jun 19 13:25 smoke_GX010143__1803d4c01a73_wasb_wasbcache
===ARTIFACTS===
C:/Users/avidu/Projects/badminton-highlight-indexer/artifacts/:
long_rally_lens_n15.json
long_rally_lens_n15.txt
multishuttle_142
pr245_code_review.md
pr245_fixes_plan.md
pr245_review.json
pr245_review.md
rally_tal_gen0

```

tridet.pdf
tridet.txt

C:/Users/avidu/Projects/badminton-highlight-indexer/artifacts/multishuttle_142:
_patch_key.json
full_f1295.png
full_f1330.png
full_f1372.png
full_f1410.png
full_f1450.png
full_f1486.png
patch_01.png
patch_02.png
patch_03.png
patch_04.png
patch_05.png
patch_06.png
patch_07.png
patch_08.png
patch_09.png
patch_10.png
patch_11.png
tracks_overlay_crop.png
tracks_overlay_full.png

C:/Users/avidu/Projects/badminton-highlight-indexer/artifacts/rally_tal_gen0:
0.1.0

C:/Users/avidu/Projects/badminton-highlight-indexer/artifacts/rally_tal_gen0/0.1.0:
manifest.json
weights.json

22. assistant (2026-06-19T16:59:53.675Z)

There's a `multishuttle\_142` artifact directory ? the empirical basis for this design. Let me examine the WASB cache structure (where det_raw.jsonl lives) and look at the GX010142 CSV to characterize the actual data. Let me also check the patch key and any multicourt analysis output.

23. user (2026-06-19T16:59:56.425Z)

```
=== wasbcache dir for 142 ===
total 3189
drwxr-xr-x 1 avidu 197609      0 Jun 19 14:10 .
drwxr-xr-x 1 avidu 197609      0 Jun 19 17:27 ..
-rw-r--r-- 1 avidu 197609 3250556 Jun 19 14:10 det_raw.jsonl
-rw-r--r-- 1 avidu 197609      310 Jun 19 14:10 manifest.json
=== patch key ===
{
  "patch_01.png": {
    "moment": 1295,
    "frames": [
      1294,
      1295,
      1296
    ],
    "loc": "prom",
    "cx": 1180.5,
    "cy": 461.3,
    "x_norm": 0.615
  },
  "patch_02.png": {
    "moment": 1295,
    "frames": [
      1294,
```

```
    1295,  
    1296  
  ],  
  "loc": "adj",  
  "cx": 1743.8,  
  "cy": 115.3,  
  "x_norm": 0.908  
},  
"patch_03.png": {  
  "moment": 1330,  
  "frames": [  
    1329,  
    1330,  
    1331  
  ],  
  "loc": "prom",  
  "cx": 904.2,  
  "cy": 261.6,  
  "x_norm": 0.471  
},  
"patch_04.png": {  
  "moment": 1330,  
  "frames": [  
    1329,  
    1330,  
    1331  
  ],  
  "loc": "adj",  
  "cx": 1672.2,  
  "cy": 207.0,  
  "x_norm": 0.871  
},  
"patch_05.png": {  
  "moment": 1372,  
  "frames": [  
    1371,  
    1372,  
    1373  
  ],  
  "loc": "prom",  
  "cx": 873.3,  
  "cy": 394.0,  
  "x_norm": 0.455  
},  
"patch_06.png": {  
  "moment": 1372,  
  "frames": [  
    1371,  
    1372,  
    1373  
  ],  
  "loc": "adj",  
  "cx": 1695.2,  
  "cy": 414.8,  
  "x_norm": 0.883  
},  
"patch_07.png": {  
  "moment": 1410,  
  "frames": [  
    1409,  
    1410,  
    1411  
  ],  
  "loc": "prom",  
  "cx": 1284.0,
```

```

        "cy": 504.5,
        "x_norm": 0.669
    },
    "patch_08.png": {
        "moment": 1410,
        "frames": [
            1409,
            1410,
            1411
        ],
        "loc": "adj",
        "cx": 1786.0,
        "cy": 171.2,
        "x_norm": 0.93
    },
    "patch_09.png": {
        "moment": 1450,
        "frames": [
            1449,
            1450,
            1451
        ],
        "loc": "adj",
        "cx": 1664.6,
        "cy": 142.6,
        "x_norm": 0.867
    },
    "patch_10.png": {
        "moment": 1486,
        "frames": [
            1485,
            1486,
            1487
        ],
        "loc": "prom",
        "cx": 1038.8,
        "cy": 636.3,
        "x_norm": 0.541
    },
    "patch_11.png": {
        "moment": 1486,
        "frames": [
            1485,
            1486,
            1487
        ],
        "loc": "adj",
        "cx": 1574.7,
        "cy": 215.7,
        "x_norm": 0.82
    }
}
=== multicourt analysis ===

```

Multicourt prominent-court gate ? quality analysis (vs golden, n=4)

	clip	n_gt	base P/R/F1	gated P/R/F1	?P	?F1	dropped (all FP?)	verdict	
	---	---	---	---	---	---	---		
	mahadevpura_2	40	0.24/0.25/0.24	0.26/0.25/0.26	+0.025	+0.013	4 (4)	marginal	
	GX010128	52	0.49/0.77/0.60	0.49/0.77/0.60	+0.000	+0.000	0 (0)	NULL	
	Badminton_BXH_2	44	0.35/0.61/0.44	0.35/0.61/0.44	+0.000	+0.000	0 (0)	NULL	
	Boxhill_Doubles	43	0.32/0.65/0.43	0.42/0.65/0.51	+0.096	+0.078	20 (20)	REAL WIN	

Synthesis

I now have everything I need, fully grounded. The Gemini profile (high precision, ~0 false positives, lower recall; G2 = precision is local's #1 gap) is confirmed at lines 41-48, and the multicourt over-fire on long rallies needing court-localization (Gate-B / court mask) is confirmed at line 48. Here is the report.

Prominent-Court Gate ? Multicourt Golden-Clip Quality Synthesis

The **prominent-court gate** (the G2 multicourt precision lever) drops detected rally windows whose shuttle mass falls majority-outside a **recall-safe polygon** containing 100% of golden-rally shuttle medians (`min_in_court_frac=0.5``). Below: per-clip results, the aggregate win, honest null/marginal flags, and a grounded "how close to Gemini" framing.

(a) Per-clip results

clip	n_gt	base P	base R	base F1	gated P	gated R	gated F1	?P	?R	?F1	#dropped	#drop-were-FP	%golden-in-poly
Boxhill_Doubles	43	0.322	0.651	0.431	0.418	0.651	0.509	0.096	0.078	20 / 20	100%	0	0
mahadevpura_2	40	0.238	0.250	0.244	0.263	0.250	0.256	0.025	0	0.013	4 / 4	100%	0
GX010128	52	0.494	0.769	0.602	0.494	0.769	0.602	0	0	0	0	0	0
Badminton_BXH_2	44	0.346	0.614	0.443	0.346	0.614	0.443	0	0	0	0	0	0

(tolF1: Boxhill 0.262-0.309; mahadevpura 0.122-0.128; GX010128 / BXH_2 unchanged.)

(b) Aggregate quality win

metric	mean across 4 clips	mean across the 2 clips that moved
?P	0.030	0.061
?F1	0.023	0.046
?R	0.000	0.000
dropped windows	24 total	?
dropped that were FP	24 / 24 (100%)	?

The signature is **precision up, recall dead-flat, every dropped window a verified false positive** ? exactly the multicourt G2 behavior the gate is designed for. The headline clip is **Boxhill_Doubles**: P **0.096** / F1 **0.078** by cutting **20** genuine adjacent-court windows (all with median shuttle $x > 0.80$, all temporally distinct from the prominent court's rallies), with the **same 28 golden rallies** matched before and after ? recall flatness verified at the matching level, not merely numerically. The polygon edge ($x = 0.80$) is the sweep-confirmed tightest recall-flat optimum, not curve-fit. This is the gate working as intended: a true spatial separation of a real second court.

(c) Honest null / marginal flags

- **GX010128** ? true NULL, not masked. Exact no-op (0 dropped). The genuine adjacent-court-looking cluster ($x > 0.87$) is a **stuck-tracker pixel-lock** (284 points at exactly $x = 1680px$ across 291 distinct seconds), not a live court ? it never forms a velocity window, so there is no FP for the gate to remove. Min in-court fraction across all 81 windows is $0.62 > 0.5$, so nothing *can* drop at threshold. No win, no harm.
- **Badminton_BXH_2** ? true NULL. No-op (0 dropped). The 51 FPs are spatially **co-located** with the true positives inside the prominent court (FP median $x[0.060, 0.497]$? TP $x[0.098, 0.483]$); ~41% of points are a static center-court tracker artifact. No separable second court exists, and every tighter polygon hits real rallies before FPs ($x > 0.48$ drops 2 TP vs 1 FP). A spatial gate structurally cannot help here.
- **mahadevpura_2** ? REAL but MARGINAL (+0.013 F1). The 4 dropped windows are all confirmed FPs (pre-first / post-last-rally and inter-rally edge fires, zero golden overlap), but the golden medians form **one central blob** with no separable secondary court ? the gate is trimming

diffuse frame-edge scatter, not performing true multicourt separation. 32/42 baseline FPs are on-prominent-court boundary/over-merge fires the gate ****cannot**** touch. Near-no-op on this clip's dominant FP mode.

****Bottom line on honesty:**** the gate delivers a clean structural win on exactly ****1 of 4**** clips (Boxhill), a marginal-but-real edge trim on 1 (mahadevpura), and ****honest** nulls on 2** (GX010128, BXH_2). ****No clip cost any recall**** (?R=0 everywhere, verified interval-for-interval), and ****no number is inflated**** ? every reported value was reproduced to ?4 decimals against the raw clips. The aggregate ?F1 is modest precisely because the gate only fires where a genuinely separable court exists; it is recall-safe by construction and silent where it has no lever.

(d) How close to Gemini

****There are no Gemini predictions for these four multicourt clips, so a direct number is impossible.**** Inventing one would violate the attribution rule. Instead, reasoning from Gemini's established corpus profile (`RALLY\_DETECTION\_GAP\_CLOSURE.md` §3, `[verified]` n=2 golden):

- Gemini's signature is ****high precision (~1.00), near-zero false positives, lower recall**** (e.g. GX010137: Gemini P=1.00 / 0 FP / R=0.79 vs local P=0.77 / 14 FP / R=0.88). ****Precision is local's #1 gap (G2)**** ? local already out-recalls Gemini on normal play, so closing the gap means ****removing false positives, not finding more rallies.**** §3 P7 specifically flags that the residual multicourt over-fire (1.58× on long rallies) needs ****court-localization (Gate-B / court mask)**** ? i.e. ***this exact gate***.

- The prominent-court gate moves local in precisely Gemini's direction ? it strips adjacent-court FPs while holding recall ? so it is directionally the right lever. Distance to a ****perfect-precision reference (P?1.0)**** after gating:

clip	gated P	residual gap to P=1.0	FP-removal progress
---	---	---	---
GX010128	0.494	**?0.506**	0 of 41 FPs removed
Badminton_BXH_2	0.346	**?0.654**	0 of 51 FPs removed
Boxhill_Doubles	0.418	**?0.582**	20 of 39 FPs removed
mahadevpura_2	0.263	**?0.737**	4 of 32 FPs removed

Even on its best clip (Boxhill), the gate removes only ****~half**** the false positives and leaves precision at ****0.418 ? a 0.58 residual gap**** to perfect precision. The remaining FPs are ****on-court**** (over-merge, rally-END over-extension G3, on-prominent-court boundary fires), which a ***spatial*** court gate cannot reach. So the gate is a ****partial down-payment**** on the G2 precision gap, not a closer of it.

- ****A real comparison requires running Gemini on these four clips and scoring it against the same golden labels**** ? that is the explicit follow-up. Until then any "X points from Gemini" figure would be fabricated. What can be stated honestly: the gate shifts local ****toward**** Gemini's failure mode (fewer FPs, recall preserved) but, on a perfect-precision yardstick, the bulk of the gap remains and is ****not spatially separable**** ? pointing to fusion / on-court precision rules (Gate-A net-crossings, R4 end-rule) as the complementary next levers.

Verdict

The prominent-court gate is a ****worthwhile but narrow**** multicourt precision lever: it is ****recall-safe by construction**** (100% golden-in-polygon, ?R=0 on every clip) and delivers a ****clean, verified structural win where a genuinely separable adjacent court exists**** (Boxhill: P +0.096, F1 +0.078, 20 real adjacent-court FPs cut). It honestly does ****nothing**** where the second "court" is a tracker artifact (GX010128) or where FPs are co-located on the prominent court (BXH_2), and only ****marginally**** trims edge scatter where the golden mass is one blob (mahadevpura). Because it is free of recall cost and zero-harm on its nulls, it is a ****safe, ship-able down-payment on G2**** ? but it closes only the ***spatially-separable*** slice of the precision gap (?half the FPs on its one strong clip), so it should land ****flag-gated/default-OFF** per the Launch-Report discipline** and be paired with on-court precision levers (Gate-A, R4 end-rule, fusion) and an actual Gemini run on these clips to quantify the true remaining distance.

Source grounded: `C:\Users\avidu\Projects\badminton-highlight-indexer\docs\RALLY\_DETECTION\_GAP\_CLOSURE.md` S3 (lines 33?48) for Gemini's profile; per-clip rows from the verified JSON provided. Note: `git pull --ff-only` failed on this branch (upstream is misconfigured to a non-existent ref `docs/court-floor-rally-end`), so this synthesis used the local checkout read-only ? no Gemini numbers were invented.

24. assistant (2026-06-19T17:00:02.384Z)

That `multicourt\_analysis.md` is gold ? it contains a stale synthesis appended by a prior agent, but the ****data table at the top is real and directly relevant****. The most important finding for my lens is line: "GX010128 ? the genuine adjacent-court-looking cluster (x?0.87) is a ****stuck-tracker pixel-lock**** (284 points at exactly x=1680px across 291 distinct seconds), not a live court ? it ****never forms a velocity window****." That's the crux. Let me verify this against the actual GX010128 data and the det_raw.jsonl structure. Let me find the GX010128 CSV and examine the stuck-pixel behavior directly.

25. user (2026-06-19T17:00:05.006Z)

```
=== find GX010128 csv ===
=== det_raw manifest 142 ===
{
  "version": 1,
  "frames_dir": "video:/home/avidullu/clips/GX010142_1080p__c4cfa94a792f.mp4",
  "weights": "/home/avidullu/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar",
  "sport": "badminton",
  "frames_in": 3,
  "frames_total": 29937,
  "windows_total": 29935,
  "windows_done": 29935
}=== det_raw first 3 lines (structure) ===
{"w":0,"f":[[0,[]],[1,[]],[2,[]]]}
{"w":1,"f":[[1,[]],[2,[]],[3,[]]]}
{"w":2,"f":[[2,[]],[3,[]],[4,[]]]}
```

26. assistant (2026-06-19T17:00:10.492Z)

GX010128 has no cached CSV locally (it's one of the owner's multicourt golden clips, processed elsewhere). The det_raw.jsonl structure is per-window with `f` = list of `[frame, [candidates]]`. Let me examine the det_raw candidate structure during the 21-25s rally region (frames ~630-750 at 30fps) to see the actual multi-candidate format, and characterize what a "track" reconstruction would look at. Let me also check what motion-energy / linker code may already exist or be planned.

27. user (2026-06-19T17:00:14.661Z)

(Bash completed with no output)

28. user (2026-06-19T17:00:22.653Z)

```
total frames: 29937 frames-with-candidate: 9313
min/max frame with cand: 44 29936
fps=30: region frames-with-cand=0, multi(>=2)=0
fps=50: region frames-with-cand=17, multi(>=2)=2
  1116 [[1316.25, 596.25, 0.6170324087142944, 0], [1323.75, 600.0, 0.5319952964782715, 0]]
  1166 [[1316.25, 594.6158409118652, 1.1655774116516113, 0]]
  1172 [[1807.500114440918, 626.3373756408691, 1.605959177017212, 0]]
  1174 [[1796.25, 622.584114074707, 1.643747329711914, 0], [1805.5638885498047,
630.000057220459, 1.0923864841461182, 0]]
fps=59.94: region frames-with-cand=206, multi(>=2)=150
  1277 [[1289.999885559082, 573.6074066162109, 1.6515653133392334, 0]]
  1281 [[1289.5104217529297, 582.5565147399902, 5.226017951965332, 0]]
  1283 [[1290.0, 588.6476898193359, 1.6576406955718994, 0]]
  1284 [[1290.0, 590.5788230895996, 1.041002869606018, 0]]
```


29. assistant (2026-06-19T17:00:27.859Z)

The clip is ~59.94 fps. The candidate format is `[x, y, score, class]`. The patch_key frames (1294-1487) line up with the 21-25s window at 59.94fps. This confirms the multi-candidate richness: 150 of 206 frames in the region carry 2 candidates. Now the critical question for my lens: **does a ghost (stuck-pixel) survive multi-track selection?** Let me reconstruct the actual behavior ? build candidate tracks and check motion energy, AND simulate the stuck-pixel scenario. Let me look at how the det_raw scores/x-distributions look, and whether a ghost would appear in det_raw vs. only being a tracker artifact.

30. user (2026-06-19T17:00:34.491Z)

Top-10 most-repeated EXACT (x,y) detector candidates (rounded .1px):

(1316.2,596.2)	x_norm=0.686	count=4
(1012.5,495.0)	x_norm=0.527	count=4
(1020.0,487.5)	x_norm=0.531	count=4
(1192.5,600.0)	x_norm=0.621	count=3
(1215.0,596.2)	x_norm=0.633	count=3
(1230.0,600.0)	x_norm=0.641	count=3
(1046.2,733.2)	x_norm=0.545	count=3
(1796.2,622.6)	x_norm=0.936	count=2
(1110.0,555.0)	x_norm=0.578	count=2
(1732.5,661.9)	x_norm=0.902	count=2

Score stats of the single most-repeated point:

pt=(1316.2,596.2) n=4 score min/med/max = 0.556/0.559/0.617

31. assistant (2026-06-19T17:00:40.015Z)

Critically important finding: **GX010142's detector (det_raw) shows NO stuck-pixel ghost** ? the max repeat of any exact (x,y) is just 4 frames. The ghost (284 frames at x=1680 in GX010128) is **not present in this clip's detector output**. This tells me the ghost is a **tracker** artifact, not a detector artifact ? which directly bears on the "subsumes the ghost" claim. Let me verify the mechanism: how does the single-target tracker produce a stuck-pixel lock, and does the windower's velocity math already kill it? Let me look for the WASB tracker code and how visibility/CSV is produced from det_raw.

32. user (2026-06-19T17:00:40.774Z)

```
1      """WASB shuttle-trajectory runner (WSL2 subprocess adapter).
2
3      Windows-side counterpart to backend/detectors/wasb_infer.py. It:
4          1. stages the video into the WSL filesystem (fast local reads, not /mnt),
5          2. copies wasb_infer.py into the WASB repo's src/ dir,
6          3. runs it in the WSL `wasb` conda env -> a Frame,Visibility,X,Y trajectory CSV,
7          4. reuses the (model-agnostic) trajectory->action-window logic from tracknet_runner.
8
9      So the only WASB-specific bits live here; the windowing + CSV parsing are shared
10     with the TrackNet path. Verified inference core: wasb_infer.py (see its docstring).
11     """
12     import os
13     import shlex
14     import logging
15     import subprocess
16     from dataclasses import dataclass
17     from typing import Dict, Any, Optional, Tuple
18
19     # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical.
20     # parse_trajectory_csv / trajectory_to_action_windows are @staticmethods on
TrackNetRunner.
21     from backend.pipeline.detectors.tracknet_runner import to_wsl_mnt_path, TrackNetRunner
22     from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
wsl_tilde_quote
23
```

```

24     logger = logging.getLogger(__name__)
25
26     # Windows path to the inference wrapper we stage into WSL.
27     _INFER_WIN = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
28
29
30     @dataclass
31     class WasbConfig:
32         """Resolved settings for a WASB WSL run (built from config.json ->
indexing.wasb)."""
33         distro: str = "Ubuntu"
34         repo_dir: str = "~/models/WASB-SBDT"
35         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36         conda_env: str = "wasb"
37         weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
38         sport: str = "badminton"
39         wsl_stage_dir: str = "~/clips"
40         timeout_sec: int = 1800 # 30 min; a hung GPU/conda call is killed (0 = no timeout)
41         keep_frames: bool = False # retain staged video + frame cache after success (debug
only)
42         min_free_gb: float = 0.0 # warn if WSL free space below this before a run (0 =
off)
43
44         # FAST PATH (default OFF until verified): decode the staged video on the fly inside
45         # WSL and feed frames straight to the detector, skipping the slow per-frame PNG
46         # extraction that dominates a long clip. Output is bit-identical to the PNG path
47         # (PNG is lossless), but this stays opt-in until the owner's side-by-side confirms
it.
48
49         stream_video: bool = False
50
51     @classmethod
52     def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "WasbConfig":
53         w = (idx_cfg or {}).get("wasb", {}) or {}
54         return cls(
55             distro=w.get("wsl_distro", "Ubuntu"),
56             repo_dir=w.get("repo_dir", "~/models/WASB-SBDT"),
57             conda_sh=w.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
58             conda_env=w.get("conda_env", "wasb"),
59             weights_path=w.get("weights_path",
~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"),
60             sport=w.get("sport", "badminton"),
61             wsl_stage_dir=w.get("wsl_stage_dir", "~/clips"),
62             timeout_sec=int(w.get("timeout_sec", 1800)),
63             keep_frames=bool(w.get("keep_frames", False)),
64             min_free_gb=float(w.get("min_free_gb", 0.0)),
65             stream_video=bool(w.get("stream_video", False)),
66         )
67
68     class WasbRunner(WslCommandMixin, DetectorRunner):
69         def __init__(self, cfg: WasbConfig):
70             self.cfg = cfg
71
72         def healthcheck(self) -> Tuple[bool, str]:
73             """Verify the WSL `wasb` env imports torch and sees a GPU. Returns (ok, msg)."""
74             cmd = (f"conda activate {self.cfg.conda_env} && "
75                 f"python -c \"import torch; print('torch', torch.__version__, "
76                 f"'cuda', torch.cuda.is_available())\"")
77             try:
78                 res = self._wsl_bash(cmd)
79             except FileNotFoundError:
80                 return False, "`wsl` not found ? Windows + WSL2 required."
81             except subprocess.TimeoutExpired:
82                 return False, "WSL healthcheck timed out."

```

```

83         out = (res.stdout or "").strip()
84         if res.returncode != 0:
85             return False, f"WSL `wasb` env check failed: {(res.stderr or out).strip()}"
86         return True, out
87
88     def _stage_video(self, video_win_path: str, dest_name: str) -> Optional[str]:
89         src_mnt = to_wsl_mnt_path(video_win_path)
90         reason = self.wsl_source_unreadable_reason(src_mnt)
91         if reason:
92             logger.error("Cannot stage video into WSL: %s", reason)
93             return None
94         dest = f"{self.cfg.wsl_stage_dir}/{dest_name}"
95         cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}"
96         {wsl_tilde_quote(dest)} && echo OK"
97         try:
98             res = self._wsl_bash(cmd)
99         except subprocess.TimeoutExpired:
100             logger.error("Staging video into WSL timed out.")
101             return None
102         if res.returncode != 0 or "OK" not in (res.stdout or ""):
103             logger.error("Failed to stage video into WSL: %s", (res.stderr or
104 res.stdout).strip())
105             return None
106         return dest
107
108     def _stage_infer_script(self) -> bool:
109         """Copy wasb_infer.py into the WASB repo src/ so it can import WASB modules."""
110         src_mnt = to_wsl_mnt_path(_INFER_WIN)
111         cmd = f"cp {shlex.quote(src_mnt)} {self.cfg.repo_dir}/src/wasb_infer.py && echo
112 OK"
113         res = self._wsl_bash(cmd)
114         return res.returncode == 0 and "OK" in (res.stdout or "")
115
116     def run_predict(self, video_win_path: str, output_win_dir: str,
117                     video_id: Optional[str] = None) -> Optional[str]:
118         """Run WASB on a video. Returns the Windows path to the trajectory CSV, or None.
119
120         All WSL/cache/output artifacts are namespaced by ``video_id`` (the file MD5) so
121         two videos that share a filename (e.g. two ``match.mp4`` from different folders)
122         can never reuse each other's staged frames, resumable cache, or CSV.
123
124         # Resolve relative dirs (the hybrids pass "output/wasb") BEFORE the /mnt
125         # translation: to_wsl_mnt_path passes relative paths through unchanged, so
126         # WSL resolved them against the inference cwd (~/.models/WASB-SBDT/src) and
127         # the CSV landed inside WSL while Windows polled the repo-relative path.
128         output_win_dir = os.path.abspath(output_win_dir)
129         os.makedirs(output_win_dir, exist_ok=True)
130         # Normalize for cross-platform basename (tests use Windows paths on Linux).
131         video_win_path = str(video_win_path).replace("\\", "/")
132         base = os.path.basename(video_win_path)
133         stem, ext = os.path.splitext(base)
134         # Unique, content-derived key: same filename + different bytes -> different key.
135         key = f"{stem}__{video_id[:12]}" if video_id else stem
136         wsl_out_dir = to_wsl_mnt_path(output_win_dir)
137         expected_csv_win = os.path.join(output_win_dir, f"{key}_wasb.csv")
138
139         if not self._stage_infer_script():
140             logger.error("Could not stage wasb_infer.py into WSL.")
141             return None
142         staged_video = self._stage_video(video_win_path, f"{key}{ext}")
143         if staged_video is None:
144             return None
145         return None
146         frames_dir = f"{self.cfg.wsl_stage_dir}/{key}_frames"
147
148         # Disk guard: a 4K clip can extract ~100 GB of PNG frames. Warn early if the

```

```

145         # WSL stage filesystem is already low so the user can clean up before it fills.
146         # (Streaming never materializes the PNGs, so the guard only matters off the fast
path.)
147         if self.cfg.min_free_gb > 0 and not self.cfg.stream_video:
148             free = self._wsl_free_gb(self.cfg.wsl_stage_dir)
149             if free is not None and free < self.cfg.min_free_gb:
150                 logger.warning(
151                     "Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f). Frame
extraction "
152                         "may fill the disk ? consider running tools/cleanup_caches.py.",
153                         free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)
154
155         # Fast path: stream-decode the video in-process (no PNG extraction). Output is
156         # bit-identical to the PNG round-trip but avoids writing ~46k PNGs for a 12-min
clip.
157         if self.cfg.stream_video:
158             frame_args = "--stream-video "
159         else:
160             frame_args = f"--frames_out_dir {wsl_tilde_quote(frames_dir)} "
161         cmd = (
162             f"conda activate {self.cfg.conda_env} && cd {self.cfg.repo_dir}/src && "
163             f"python wasb_infer.py "
164             f"--video {wsl_tilde_quote(staged_video)} "
165             f"{frame_args}"
166             f"--weights {wsl_tilde_quote(self.cfg.weights_path)} "
167             f"--sport {shlex.quote(self.cfg.sport)} "
168             f"--out {wsl_tilde_quote(wsl_out_dir + '/' + key + '_wasb.csv')}}")
169
170         logger.info("Running WASB inference on %s ...", base)
171         try:
172             res = self._wsl_bash(cmd)
173         except subprocess.TimeoutExpired:
174             logger.error("WASB inference timed out after %ss.", self.cfg.timeout_sec)
175             return None
176         if res.returncode != 0:
177             logger.error("WASB inference failed:\n%s", (res.stderr or
res.stdout)[-2000:])
178             return None
179         if not os.path.exists(expected_csv_win):
180             logger.error("WASB finished but expected CSV not found at %s",
expected_csv_win)
181             return None
182         logger.info("WASB trajectory CSV: %s", expected_csv_win)
183
184         # Success ? the trajectory CSV is the durable output; the staged video copy and
185         # the (huge) decoded frame cache are now pure intermediates. Delete them unless
186         # explicitly retained. On any earlier failure we return above WITHOUT cleaning,
187         # so the frame cache + detector cache remain for a resumable re-run.
188         # (Streaming never writes a frame cache, so there's only the staged video to
drop.)
189         if not self.cfg.keep_frames:
190             stale = [staged_video] if self.cfg.stream_video else [frames_dir,
staged_video]
191             logger.info("Cache hygiene: removing staged video%s for %s "
192                         "(set indexing.wasb.keep_frames=true to retain).",
193                         "" if self.cfg.stream_video else " + frame cache", key)
194             self._wsl_rm_rf(stale)
195             return expected_csv_win
196
197         # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).
198         parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
199         trajectory_to_action_windows =
staticmethod(TrackNetRunner.trajectory_to_action_windows)
200

```

33. assistant (2026-06-19T17:00:45.535Z)

The tracker logic lives in `wasb\_infer.py` inside WSL (not in this repo's git tree ? it's staged into the WASB repo). Let me check if there's a copy here and look at the online tracker, since that's where the stuck-pixel lock originates. Let me also verify the key claim about the GX010128 ghost from the multicourt analysis ? it never forms a velocity window ? by examining the windower math against a synthetic stuck-pixel trace.

34. user (2026-06-19T17:00:46.111Z)

```
.claude\worktrees\bhi-execpolicy\backend\pipeline\detectors\wasb_infer.py
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\pipeline\detectors\wasb_infer.py
backend\pipeline\detectors\wasb_infer.py
.claude\worktrees\tier1-windowing\backend\pipeline\detectors\wasb_infer.py
.claude\worktrees\w2-safeguard\backend\pipeline\detectors\wasb_infer.py
.claude\worktrees\hardchop\backend\pipeline\detectors\wasb_infer.py
.claude\worktrees\boxhill117\backend\pipeline\detectors\wasb_infer.py
.claude\worktrees\pr-stitch\backend\pipeline\detectors\wasb_infer.py
.claude\worktrees\gap-doc\backend\pipeline\detectors\wasb_infer.py
.claude\worktrees\resweep\backend\pipeline\detectors\wasb_infer.py
.claude\worktrees\recdoc\backend\pipeline\detectors\wasb_infer.py
```

35. user (2026-06-19T17:00:47.716Z)

Found 1 file
backend\pipeline\detectors\wasb_infer.py

36. user (2026-06-19T17:00:52.062Z)

```
backend\pipeline\detectors\wasb_infer.py:8:      Frame,Visibility,X,Y
backend\pipeline\detectors\wasb_infer.py:28:      the full ordered cache -> trajectory CSV. So a
lost trajectory CSV costs seconds,
backend\pipeline\detectors\wasb_infer.py:107:def _dets_to_tracker(plain_list):
backend\pipeline\detectors\wasb_infer.py:198:          w.writerow(["Frame", "Visibility", "X",
"Y"])
backend\pipeline\detectors\wasb_infer.py:299:      frame indistinguishable from the on-disk one to
everything downstream.
backend\pipeline\detectors\wasb_infer.py:348:      # restore on exit. (Plain assignment, not
setattr-with-constant, to stay ruff B010-clean.)
backend\pipeline\detectors\wasb_infer.py:349:      _dl.read_image =
_make_streamed_read_image(store, real_read_image) # type: ignore[assignment]
backend\pipeline\detectors\wasb_infer.py:353:      _dl.read_image = real_read_image # type:
ignore[assignment]
backend\pipeline\detectors\wasb_infer.py:417:      annos = [{"frame_path": p, "center":
Center(is_visible=False, x=-1.0, y=-1.0)} for p in window]
backend\pipeline\detectors\wasb_infer.py:538:      rows.append((fid, 1 if r.get("visi") else
0, r.get("x", -1), r.get("y", -1)))
backend\pipeline\detectors\wasb_infer.py:541:      visible = sum(1 for _, v, *_ in rows if v)
backend\pipeline\detectors\wasb_infer.py:543:      logger.info(f"visible-shuttle frames:
{visible}/{len(rows)}")
```

37. user (2026-06-19T17:00:55.184Z)

```
100
101     def _det_plain(det) -> list:
102         """A detector candidate dict {'xy': np.array([x,y]), 'score', 'scale'} -> JSON-able
list."""
103         xy = det["xy"]
104         return [float(xy[0]), float(xy[1]), float(det["score"]), int(det["scale"])]
105
106
107     def _dets_to_tracker(plain_list):
108         """Inverse of _det_plain: rebuild the dicts the tracker expects (xy MUST be a numpy
array,
```

```

109         the tracker does vector math / np.linalg.norm on it)."""
110         import numpy as np
111         return [{"xy": np.array([p[0], p[1]], dtype=float), "score": p[2], "scale": p[3]}
112                 for p in plain_list]
113
114
115     def write_manifest(cache_dir: str, manifest: dict) -> None:
116         _, mpath = _cache_paths(cache_dir)
117         _atomic_write_text(mpath, json.dumps(manifest, indent=2))
118
119
120     def read_manifest(cache_dir: str):
121         _, mpath = _cache_paths(cache_dir)
122         if not osp.exists(mpath):
123             return None
124         try:
125             with open(mpath) as f:
126                 return json.load(f)
127         except (json.JSONDecodeError, OSError):
128             return None
129
130
131     def manifest_compatible(manifest: dict, *, frames_dir: str, weights: str, sport: str,
132                             frames_in: int, frames_total: int) -> bool:
133         """A cache is reusable only if the run that produced it matches this run's
134         inputs."""
135         if not manifest or manifest.get("version") != CACHE_VERSION:
136             return False
137         return (
138             manifest.get("frames_dir") == osp.abspath(frames_dir)
139             and manifest.get("weights") == weights
140             and manifest.get("sport") == sport
141             and manifest.get("frames_in") == frames_in
142             and manifest.get("frames_total") == frames_total
143         )
144
145     def reset_cache(cache_dir: str) -> None:
146         """Drop any existing cache so the next run starts clean."""
147         det_jsonl, mpath = _cache_paths(cache_dir)
148         for p in (det_jsonl, mpath):
149             if osp.exists(p):
150                 os.remove(p)
151
152
153     def load_and_clean_cache(cache_dir: str):
154         """Read the longest *contiguous* (w == line index) valid prefix of det_raw.jsonl,
155         rebuild the per-frame detection accumulation, and rewrite the file to exactly that
156         prefix so a trailing half-written line from a crash can't corrupt future appends.
157
158         Returns (windows_done, det_by_fid) where det_by_fid maps frame_id -> list of
159         [x, y, score, scale] candidates (accumulated across every window the frame is in,

```

38. assistant (2026-06-19T17:00:58.319Z)

Now let me see the tracker selection logic ? specifically how WASB's online tracker picks one candidate per frame (the part that produces the stuck-pixel lock). Let me look at the section that builds the trajectory from det candidates.

39. user (2026-06-19T17:00:59.120Z)

```

380         frames dir); the streaming path passes a stable ``video:<abspath>`` key so a resume
381         after reboot still matches.
382         """
383         import time

```

```

384 import torch
385 from torch.utils.data import DataLoader
386 from detectors import build_detector
387 from trackers import build_tracker
388 from dataloaders import build_img_transforms, build_seq_transforms
389 from dataloaders.dataset_loader import ImageDataset
390 from utils import Center
391
392 streaming = frame_loader is not None
393
394 cfg = _build_cfg(weights, sport)
395 frames_in = int(cfg["model"]["frames_in"])
396 input_wh = (int(cfg["model"]["inp_width"]), int(cfg["model"]["inp_height"]))
397 output_wh = (int(cfg["model"]["out_width"]), int(cfg["model"]["out_height"]))
398
399 if streaming:
400     # `frames` is already the ordered list of (synthetic) frame paths.
401     if not isinstance(frames, (list, tuple)):
402         raise SystemExit("streaming mode requires `frames` to be a list of frame
paths")
403     frames = list(frames)
404     frames_dir = source_key or "stream"
405 else:
406     frames_dir = frames
407     frames = sorted(glob.glob(osp.join(frames_dir, "*.png"))) +
glob.glob(osp.join(frames_dir, "*.jpg")))
408     if limit:
409         frames = frames[:limit]
410     if len(frames) < frames_in:
411         where = frames_dir if not streaming else "video stream"
412         raise SystemExit(f"need >= {frames_in} frames, found {len(frames)} in {where}")
413
414     # Sliding windows of `frames_in` consecutive frames (the unit of GPU work +
checkpoint).
415     samples = []
416     for window in build_windows(frames, frames_in):
417         annos = [{"frame_path": p, "center": Center(is_visible=False, x=-1.0, y=-1.0)}
for p in window]
418         samples.append({"frames": window, "annos": annos})
419     windows_total = len(samples)
420
421     # ---- cache setup / resume decision ----- #
422     if cache_dir is None:
423         cache_dir = default_cache_dir(out_csv)
424         os.makedirs(cache_dir, exist_ok=True)
425         det_jsonl, _ = _cache_paths(cache_dir)
426         cache_key = source_key if source_key is not None else osp.abspath(frames_dir)
427         manifest = None if fresh else read_manifest(cache_dir)
428         resume = manifest_compatible(
429             manifest or {}, frames_dir=cache_key, weights=weights, sport=sport,
430             frames_in=frames_in, frames_total=len(frames),
431         )
432     if resume:
433         windows_done, det_by_fid = load_and_clean_cache(cache_dir)
434         logger.info(f"resuming from cache: {windows_done}/{windows_total} windows
already detected")
435     else:
436         if manifest is not None:
437             logger.info("cache incompatible with this run -> starting fresh")
438             reset_cache(cache_dir)
439             windows_done, det_by_fid = 0, defaultdict(list)
440
441     base_manifest = {
442         "version": CACHE_VERSION,
443         "frames_dir": cache_key,

```

```

444         "weights": weights,
445         "sport": sport,
446         "frames_in": frames_in,
447         "frames_total": len(frames),
448         "windows_total": windows_total,
449         "windows_done": windows_done,
450     }
451     write_manifest(cache_dir, base_manifest)
452
453     # ---- detector pass over the un-cached windows ----- #
454     remaining = samples[windows_done:]
455     if remaining:
456         _, transform_test = build_img_transforms(cfg)
457         try:
458             _, seq_transform_test = build_seq_transforms(cfg)
459         except Exception:
460             seq_transform_test = None
461         ds = ImageDataset(cfg, remaining, input_wh, output_wh,
462                          transform=transform_test, seq_transform=seq_transform_test,
463                          is_train=False)
464         # Streaming forces single-process loading: the in-memory frame store + the
465         # cv2.imread shim live in THIS process and can't be pickled to DataLoader
466         workers.
467         loader_workers = 0 if streaming else num_workers
468         loader = DataLoader(ds, batch_size=batch_size, shuffle=False,
469                             num_workers=loader_workers)
470         detector = build_detector(cfg)
471
472         from contextlib import ExitStack
473
474         t0 = time.time()
475         done = windows_done
476         win_idx = windows_done
477         log_every = max(1, log_every_batches)
478         flush_n = max(1, flush_every)
479         pending = []
480         logger.info(f"detecting on {len(remaining)} remaining windows "
481                    f"(total {windows_total}, batch={batch_size},
482                    workers={loader_workers}, "
483                    f"source={'video-stream' if streaming else 'frames-dir'})...")
484         det_f = open(det_jsonl, "a")
485         try:
486             with ExitStack() as stack:
487                 stack.enter_context(torch.no_grad())
488                 # In streaming mode, route ImageDataset's per-frame `cv2.imread(path)`
489                 # to the
490                 # in-memory decoded frame for that synthetic path; every other line of
491                 # __getitem__ runs unchanged so the tensor matches the PNG round-trip
492                 exactly.
493                 # The detector's first needed frame is `windows_done` (window i starts
494                 at
495                 # frame i), so prime the store there for a resume.
496                 if streaming:
497                     stack.enter_context(
498                         _streaming_read_image_patch(frame_loader, len(frames),
499                                                       window=frames_in,
500                                                       start_index=windows_done))
501                 for bi, batch in enumerate(loader):
502                     imgs, _hms, trans = batch[0], batch[1], batch[2]
503                     img_paths = [list(t) for t in batch[-1]] # [frame_in][batch] ->
504                     path
505                     batch_results, _ = detector.run_tensor(imgs, trans)
506                     nb = imgs.shape[0]
507                     for ib in range(nb):
508                         frames_payload = []

```



```

500         for ie in sorted(batch_results[ib].keys()):
501             p = img_paths[ie][ib]
502             fid = _frame_id(p)
503             plain = [_det_plain(d) for d in batch_results[ib][ie]]
504             frames_payload.append([fid, plain])
505             det_by_fid[fid].extend(plain)
506         pending.append({"w": win_idx, "f": frames_payload})
507         win_idx += 1
508     done += nb
509     if len(pending) >= flush_n or done >= windows_total:
510         _append_windows(det_f, pending)
511         pending = []
512         base_manifest["windows_done"] = done
513         write_manifest(cache_dir, base_manifest)
514     if (bi + 1) % log_every == 0 or done >= windows_total:
515         el = time.time() - t0
516         new_done = done - windows_done
517         rate = new_done / el if el > 0 else 0.0
518         eta = (windows_total - done) / rate if rate > 0 else 0.0
519         logger.info(f"{done}/{windows_total} "
520                     f"{{100 * done // max(1, windows_total)}}% | "
521                     f"elapsed {{el:.0f}}s | ETA {{eta:.0f}}s")
522     if pending:
523         _append_windows(det_f, pending)
524         base_manifest["windows_done"] = done
525         write_manifest(cache_dir, base_manifest)
526     finally:
527         det_f.close()
528     logger.info(f"detection done in {time.time() - t0:.0f}s; running tracker...")
529 else:
530     logger.info("detector cache already complete -> tracker-only re-run")
531
532 # ---- tracker re-run over the full ordered cache (cheap, deterministic) ---- #
533 tracker = build_tracker(cfg)
534 tracker.refresh()
535 rows = []
536 for fid in sorted(det_by_fid.keys()):
537     r = tracker.update(_dets_to_tracker(det_by_fid[fid]))
538     rows.append((fid, 1 if r.get("visi") else 0, r.get("x", -1), r.get("y", -1)))
539

```

40. assistant (2026-06-19T17:01:05.067Z)

Now I understand the full architecture. The tracker (`build\_tracker(cfg)` from the WASB repo's `trackers` module) is what collapses candidates to one (x,y) per frame and is the source of the stuck-pixel lock. That code lives in the upstream WASB repo (not in this tree). The key insight: the proposed Layer-1 multi-track linker operates over `det\_by\_fid` (the cached candidates), bypassing the WASB tracker entirely.

I now have everything I need to reason rigorously. Let me run one decisive empirical check: simulate the windower against a synthetic stuck-pixel trace to confirm/refute the "ghost never forms a velocity window" claim, and verify how a stuck-pixel would behave as a *track* under motion-energy selection.

41. user (2026-06-19T17:01:16.321Z)

```

A pure-stuck windows: []
B jittery-stuck windows: [{'start': 16.700033366700033, 'end': 21.40473807140474,
'active_frame_count': 271, 'peak_velocity': 0.22721359915714792, 'mean_velocity':
0.10450323700532085, 'track_id_count': 1, 'chunk_index': None}]
    per-frame velocity of a 3px jump @ 59.94fps = 0.0937 (thresh=0.02)
    px-jump needed to exceed vt=0.02: 0.64px/frame
C teleport(flip-flop) windows: [{'start': 16.700033366700033, 'end': 21.40473807140474,
'active_frame_count': 283, 'peak_velocity': 22.7896875, 'mean_velocity': 22.7896875,

```

```
'track_id_count': 1, 'chunk_index': None}]
```

42. assistant (2026-06-19T17:01:24.830Z)

This is a decisive empirical finding. The claim "a stuck pixel never forms a velocity window" is **only true for a perfectly-still pixel (Scenario A)**. With realistic sub-pixel jitter (Scenario B: $\pm 3\text{px}$), it produces a 271-frame "active" window with mean velocity 0.10 (5 $\times$ threshold) ? a robust false rally. At 59.94fps, a jump of just **0.64px/frame** exceeds the velocity threshold. So the "exactly $x=1680$ " ghost is a best case; a jittery ghost is a worse, common case.

This directly attacks the motion-energy subsumption claim: **motion energy alone does NOT distinguish a jittery ghost from a real rally.** Let me confirm one more angle ? whether a jittery ghost passes Gate-A net-crossings (the other selection criterion), and whether it sits inside a plausible court polygon.

43. user (2026-06-19T17:01:30.711Z)

```
Gate-A on tight jittery ghost: [(131, True)]
  play-axis=x net=1680.1 span=14.5 (deadband=0.87px)
Gate-A on random-walk ghost: [(10, True)] span=138.7
```

44. assistant (2026-06-19T17:01:39.503Z)

Critical confirmation. **Gate-A is actively fooled by a jittery ghost.** Because Gate-A estimates the net position as the *median of that track's own points* (the ghost's own center), a tight jittery ghost crosses its own median **131 times** ? vastly exceeding the `min_crossings=2` threshold. Gate-A's net-crossing logic was designed for a real bimodal rally (shuttle at two ends), but a stationary jitter cluster has its median *in the middle of the jitter*, so noise crossings inflate the count. The deadband ($0.06 \times \text{span} = 0.87\text{px}$) is too small to suppress jitter that exceeds 1px.

I now have all the evidence I need. Let me do a final check on the "ghost inside the polygon" edge case ? confirm a center-court ghost (Badminton_BXH_2, ~41% parked at court-center) is by definition inside the prominent-court polygon.

The Badminton_BXH_2 data point from the multicourt analysis already confirms this: the static center-court artifact has median x in $[0.060, 0.497]$, co-located with true positives inside the prominent court. A court-membership score gives the center ghost a **perfect** in-court score. So neither selection criterion (court-membership nor motion-energy-via-jitter) excludes it.

I have a complete, grounded picture. Let me compile the structured verdict.

45. user (2026-06-19T17:02:45.358Z)

Structured output provided successfully

46. assistant (2026-06-19T17:03:00.142Z)

I've completed the adversarial review and returned my verdict via StructuredOutput.

Summary of what I found (all grounded in code + cached data, not the design narrative):

Verdict: design-needs-revision (not broken ? the flaws are mitigable, but the "subsumes the ghost" claim as written is false).

The decisive empirical findings:

- The premise "ghost has ~0 motion energy ? never selected" holds **only for a bit-exactly-still pixel**. I reproduced against the production windower (`tracknet_runner.py`) that a $\pm 3\text{px}$ **jittery** lock produces a 271-frame active window at mean velocity 0.10 (5 $\times$ threshold). At the corpus's ~60fps, a 0.64px/frame jump already clears `velocity_thresh=0.02`.
- **Gate-A is fooled, not protective**: a tight jittery ghost scores **131 net-crossings** (kept=True) because Gate-A estimates the net from the track's *own* median. The second "rally-likeness" pillar green-lights the ghost.

- ****Center-court ghost defeats court-membership by construction****:
`output/multicourt\_analysis.md` shows Badminton_BXH_2's ghost (~41% parked at center) sits inside the prominent-court polygon ? perfect court score. Both selection pillars reward it.
- ****Root-cause is distinct****: GX010142's `det\_raw.jsonl` shows no detector candidate repeats beyond 4 frames, so the 284-frame lock is a ****tracker-coasting**** artifact, not a detector blob. The multi-track linker (default-OFF, over candidates) lives in a different code path than the single-track tracker (production default) that actually creates the shipped ghost.

The real discriminator the design **meant** but mis-specified is spatial ****dispersion**** (low radius-of-gyration), not motion ****energy**** ? plus a per-candidate score floor (candidates carry ~0.55 confidence). Those are cheap, pure-Python, orthogonal levers.

Note: `git pull --ff-only` failed ? the branch `docs/court-floor-rally-end` tracks a remote ref that wasn't fetched (likely deleted upstream). I worked read-only on the local checkout; flagging in case the tracking config needs fixing.

Relevant files: `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\detectors\tracknet\_runner.py` (windower velocity math),
`backend\pipeline\detectors\rally\_gate.py` (Gate-A net-crossings),
`backend\pipeline\detectors\court\_gate.py` (existing subtractive court filter),
`backend\pipeline\detectors\wasb\_infer.py:533` (tracker re-run that produces the ghost),
`output/multicourt\_analysis.md` (the GX010128/BXH_2 ghost characterization),
`docs\PROMINENT\_COURT\_DETECTION.md` (the design doc to revise).